

마이크로프로세서

Chapter 5. ADC



Human-Robot Interaction
Laboratory



KYUNG HEE
UNIVERSITY

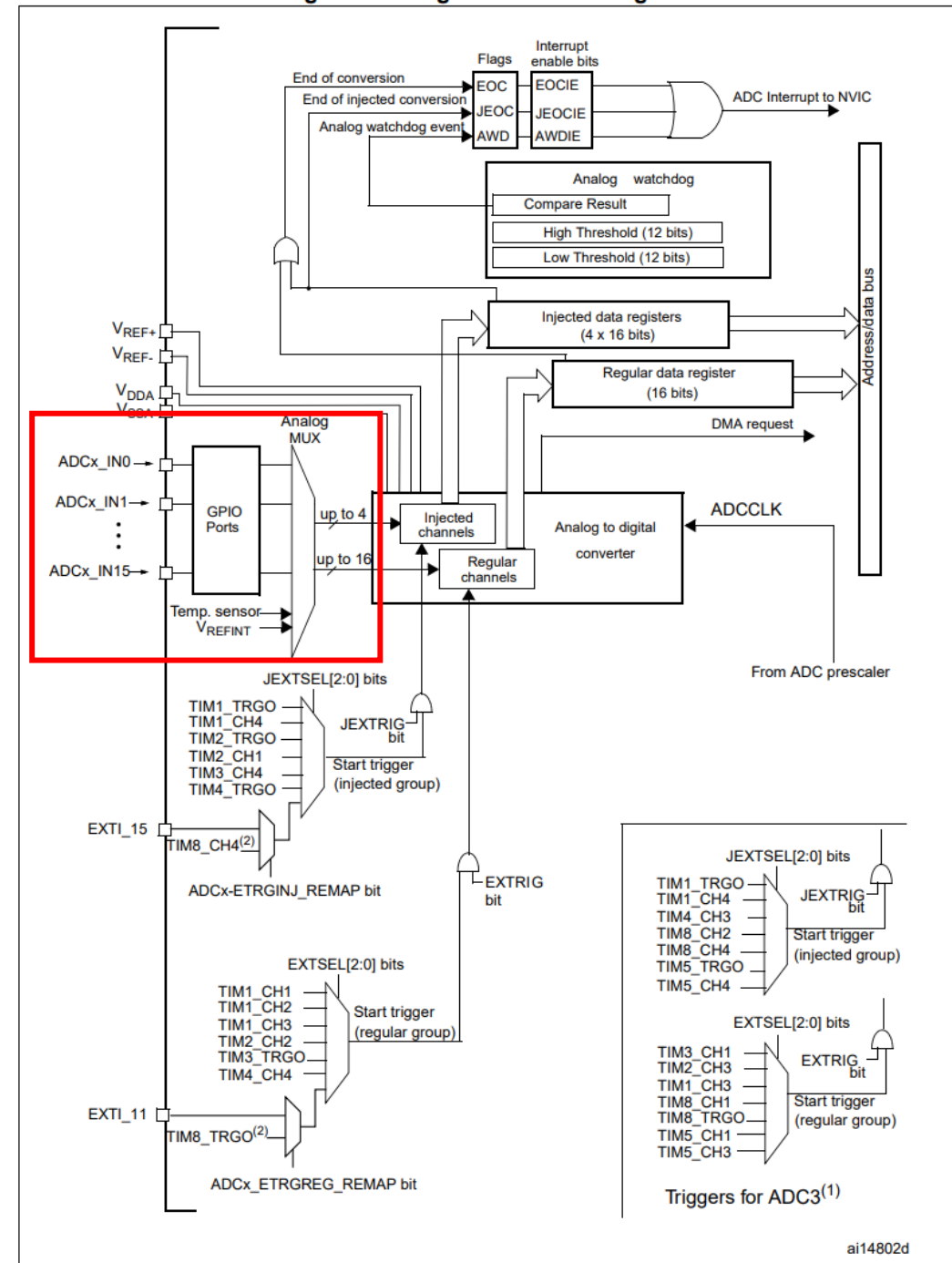
>> Analog to Digital Converter

- STM32F103RB는 12비트 해상도의 ADC 지원
- 변환 완료(End of conversion), 인젝션 변환 완료(End of Injected conversion), Analog watchdog 이벤트에서 인터럽트 생성
- 단일 변환, 연속 변환 모드 가능
- 채널 0번부터 N번까지 순차적으로 변환 가능한 스캔모드 지원과 자체 캘리브레이션 기능
- 채널별 개별적인 샘플링 시간 설정
- 2개의 ADC를 활용한 듀얼 모드
- 입력 전압 범위: $V_{REF-} \leq V_{IN} \leq V_{REF+}$

>> Analog to Digital Converter

- **Analog MUX: GPIO 포트중에서 ADC 입력 채널 하나를 선택**
- **Analog to Digital Converter: 12비트 해상도, 0~4095까지의 디지털 데이터를 통해 아날로그 신호 변환**
- **Regular Conversion: 일반 변환은 순차적으로 ADC를 실행하여 데이터 레지스터에 데이터를 덮어 쓰며 저장, 최대 16채널 변환 가능**
- **Injection Conversion: 인젝션 변환은 높은 우선순위를 가져, 기존에 진행중인 일반 변환을 중단하고 먼저 변환을 진행, 4개의 데이터 레지스터를 활용함, 최대 4개 채널 변환 지원**

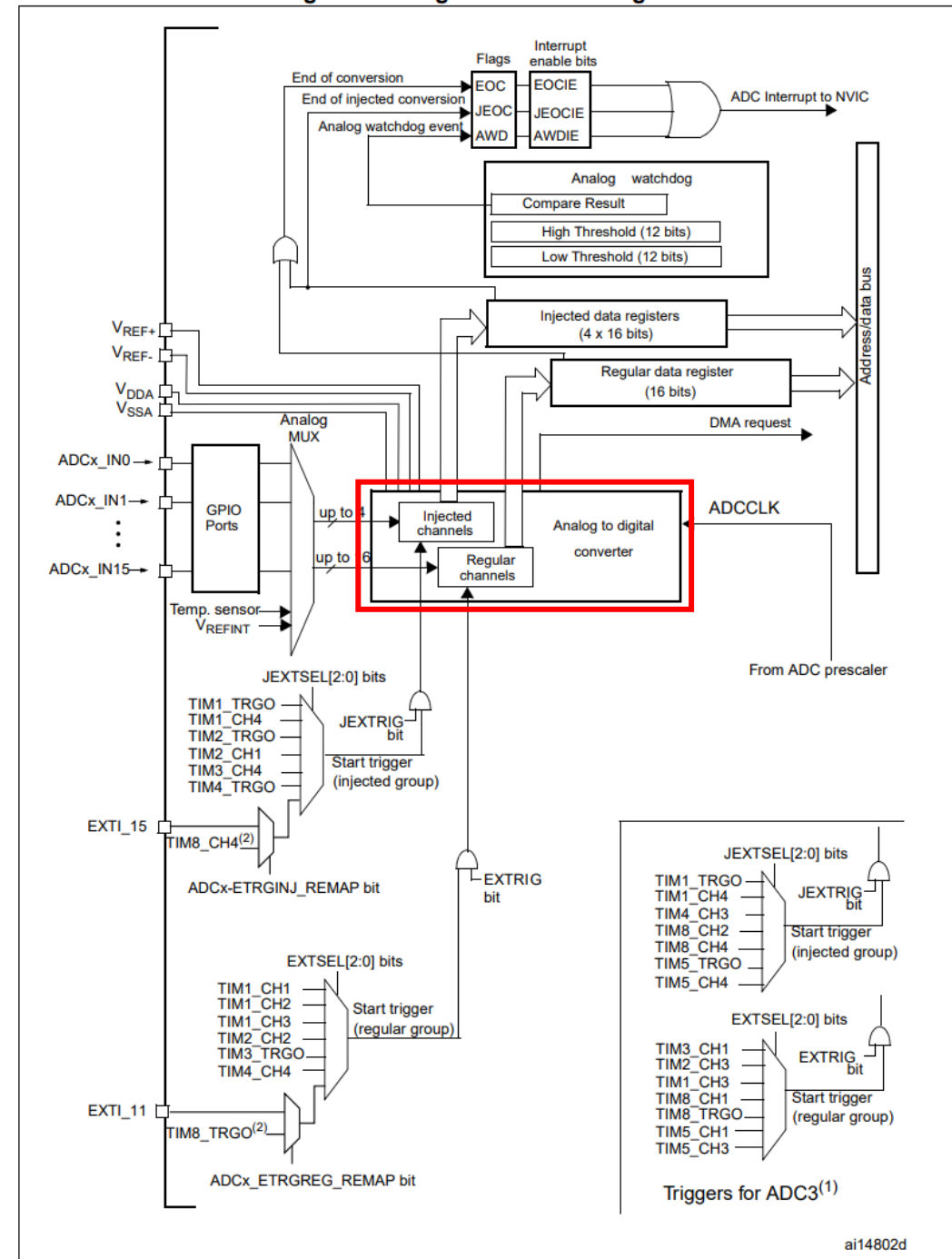
Figure 22. Single ADC block diagram



>> Analog to Digital Converter

- **Analog MUX:** GPIO 포트중에서 ADC 입력 채널 하나를 선택
- **Analog to Digital Converter:** 12비트 해상도, 0~4095까지의 디지털 데이터를 통해 아날로그 신호 변환
- **Regular Conversion:** 일반 변환은 순차적으로 ADC를 실행하여 데이터 레지스터에 데이터를 덮어 쓰며 저장, 최대 16채널 변환 가능
- **Injection Conversion:** 인젝션 변환은 높은 우선순위를 가져, 기존에 진행중인 일반 변환을 중단하고 먼저 변환을 진행, 4개의 데이터 레지스터를 활용함, 최대 4개 채널 변환 지원

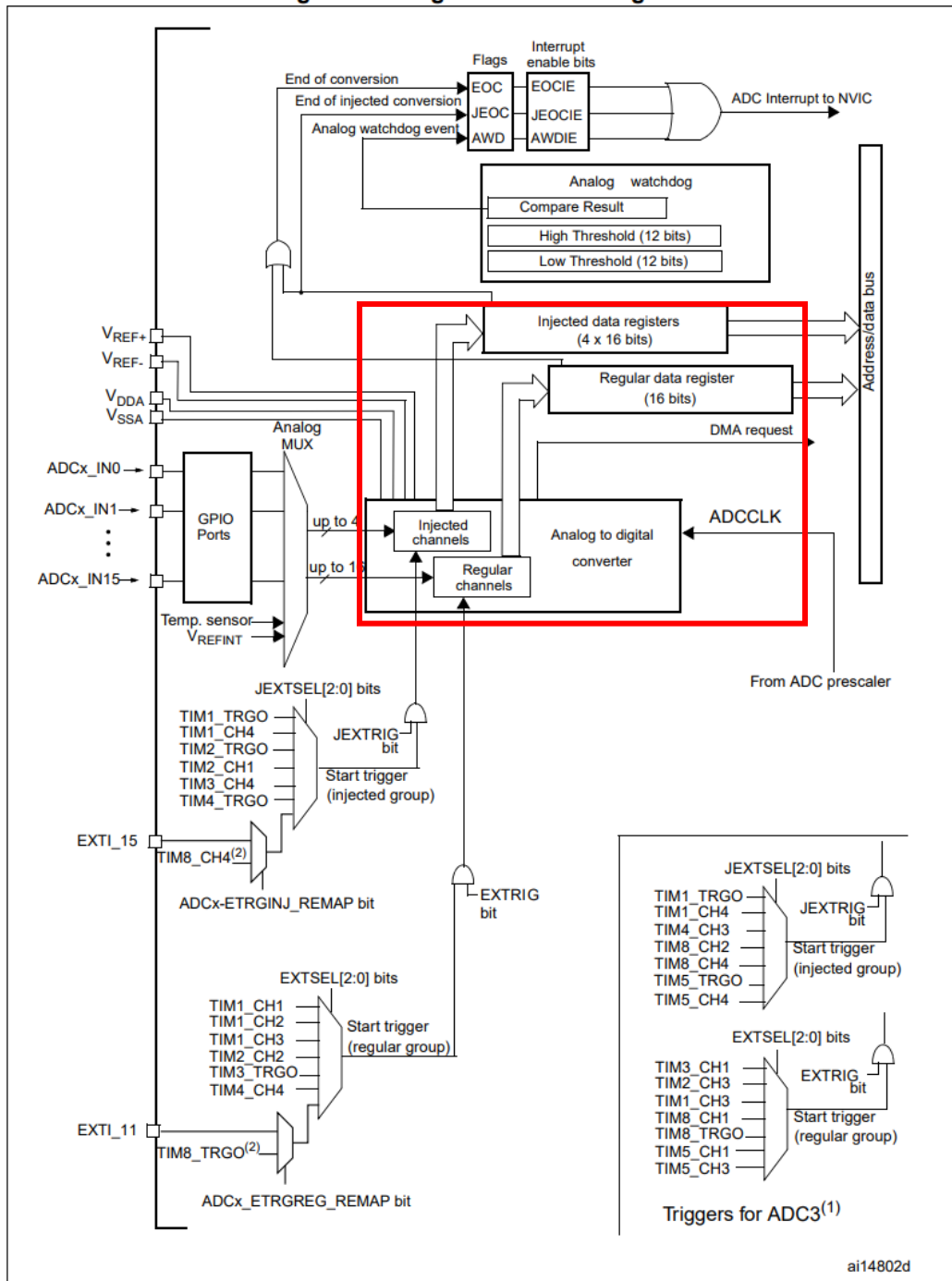
Figure 22. Single ADC block diagram



>> Analog to Digital Converter

- **Analog MUX:** GPIO 포트중에서 ADC 입력 채널 하나를 선택
- **Analog to Digital Converter:** 12비트 해상도, 0~4095까지의 디지털 데이터를 통해 아날로그 신호 변환
- **Regular Conversion:** 일반 변환은 순차적으로 ADC를 실행하여 데이터 레지스터에 데이터를 덮어 쓰며 저장, 최대 16채널 변환 가능
- **Injection Conversion:** 인젝션 변환은 높은 우선순위를 가져, 기존에 진행중인 일반 변환을 중단하고 먼저 변환을 진행, 4개의 데이터 레지스터를 활용함, 최대 4개 채널 변환 지원

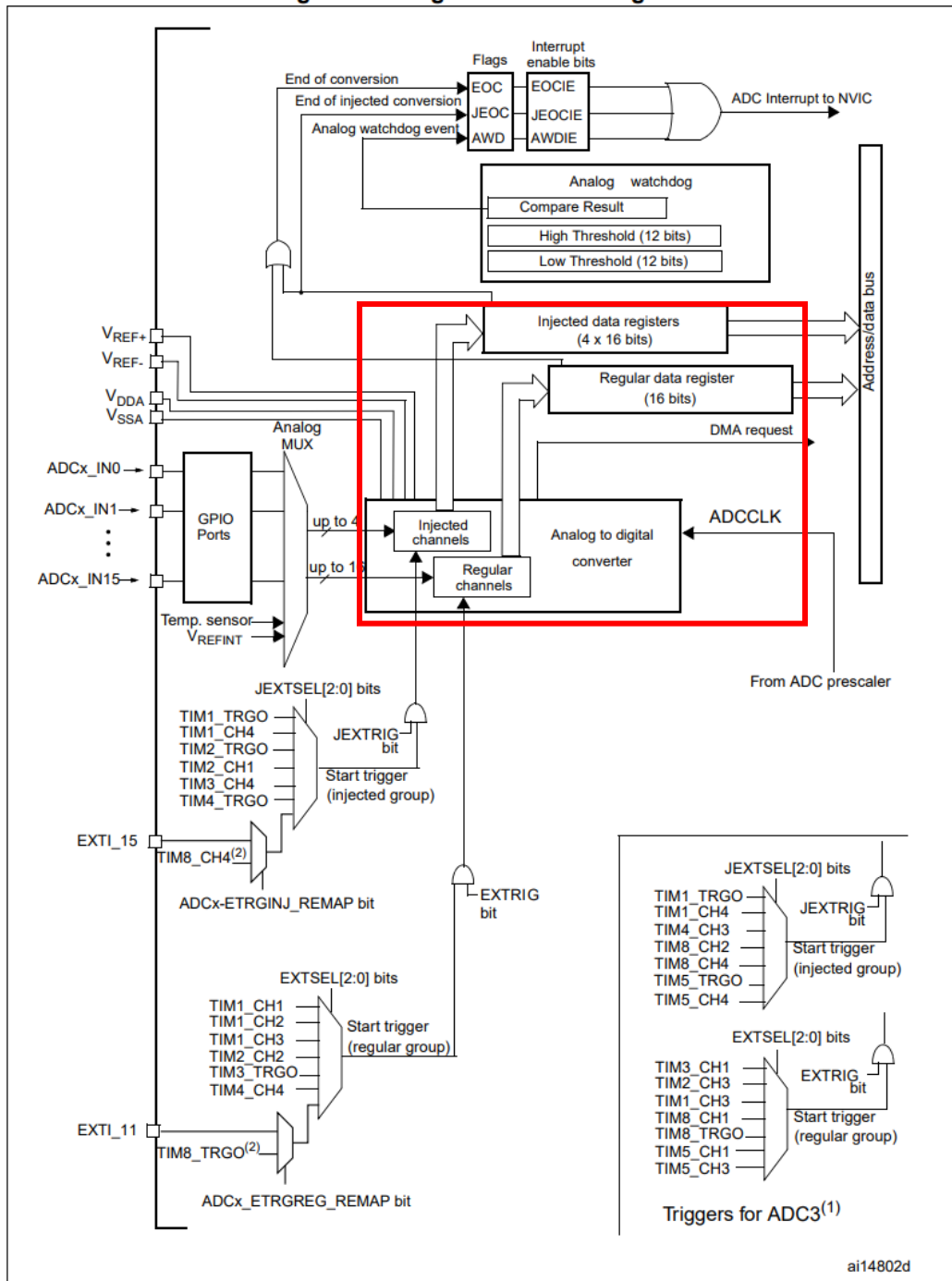
Figure 22. Single ADC block diagram



>> Analog to Digital Converter

- **Analog MUX:** GPIO 포트중에서 ADC 입력 채널 하나를 선택
- **Analog to Digital Converter:** 12비트 해상도, 0~4095까지의 디지털 데이터를 통해 아날로그 신호 변환
- **Regular Conversion:** 일반 변환은 순차적으로 ADC를 실행하여 데이터 레지스터에 데이터를 덮어 쓰며 저장, 최대 16채널 변환 가능
- **Injection Conversion:** 인젝션 변환은 높은 우선순위를 가져, 기존에 진행중인 일반 변환을 중단하고 먼저 변환을 진행, 4개의 데이터 레지스터를 활용함, 최대 4개 채널 변환 지원

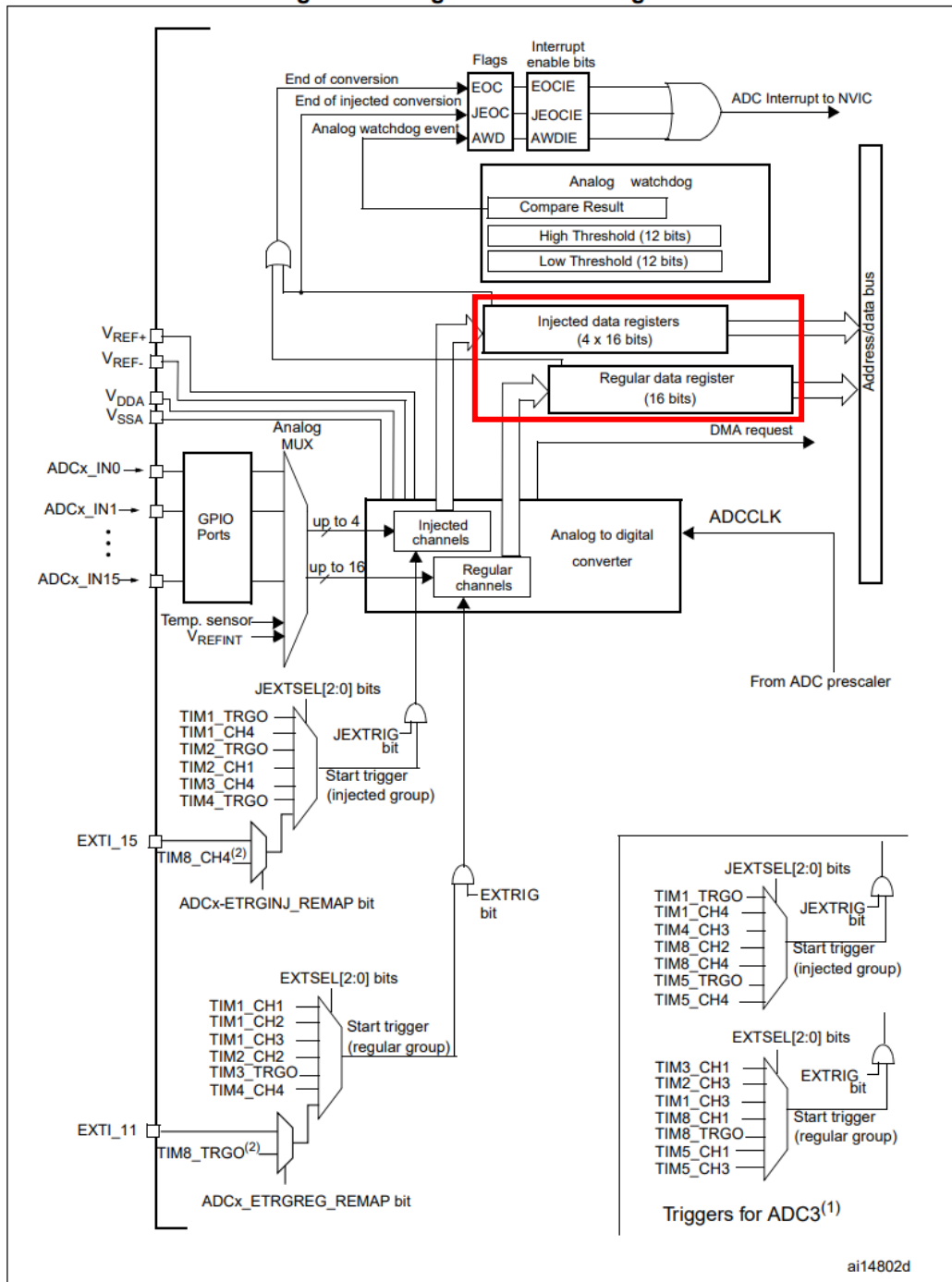
Figure 22. Single ADC block diagram



>> Analog to Digital Converter

- Data Register: 변환된 데이터 저장
- ADC Interrupt: End of conversion, End of injection conversion, Analog watchdog를 지원, 변환 종료와 상/하한 범위 이탈 인터럽트 발생
- V_{REF} : 참조 전압, STM32F103RB는 아날로그 공급 전원 V_{DDA} 와 V_{SSA} 에 연결되어 있음
- EXTRIG, JEXTRIG: ADC 변환 시작을 위한 외부 트리거, 타이머, GPIO, 인터럽트를 통해 ADC 변환 제어가 가능

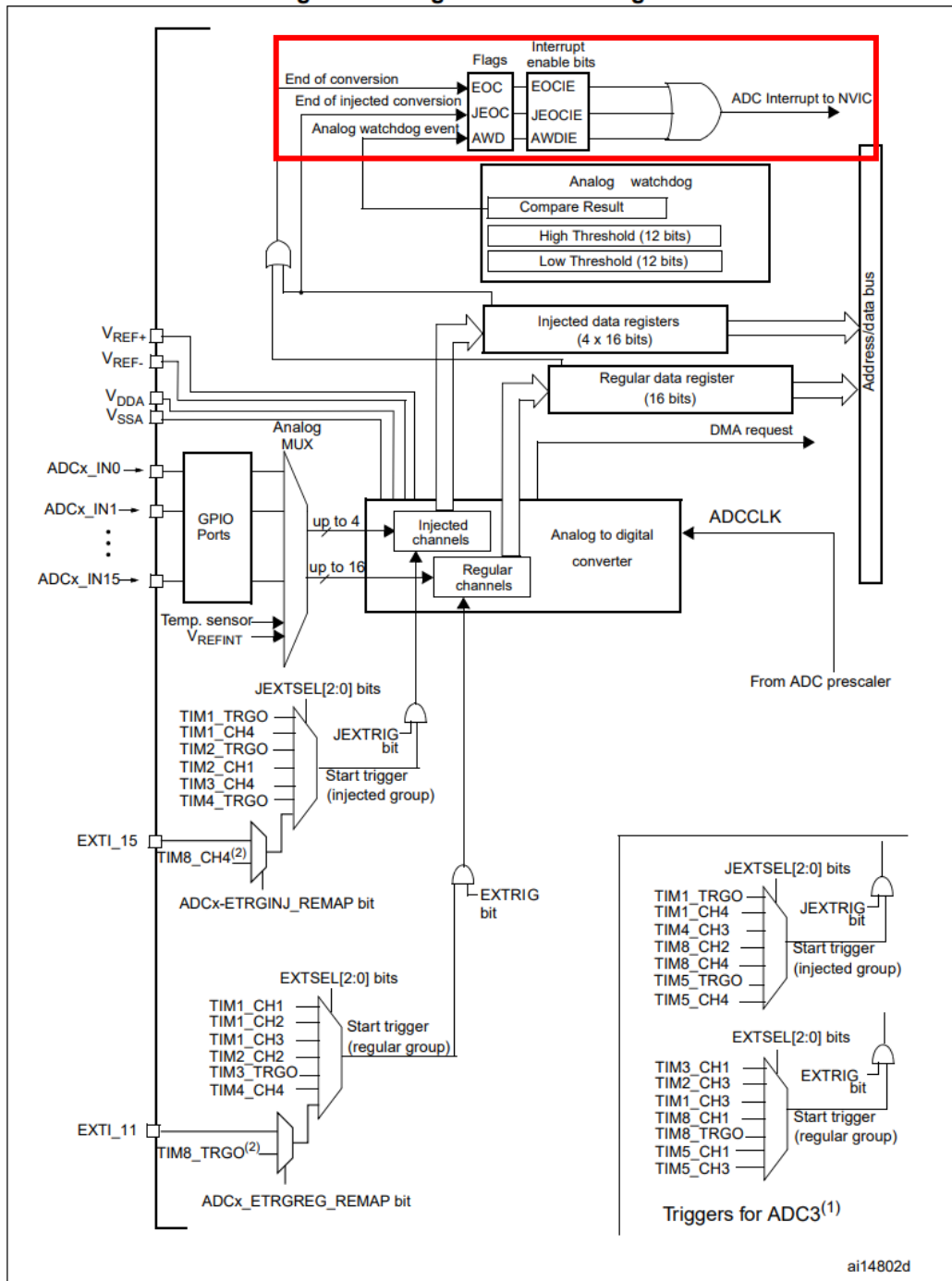
Figure 22. Single ADC block diagram



>> Analog to Digital Converter

- Data Register: 변환된 데이터 저장
- ADC Interrupt: End of conversion, End of injection conversion, Analog watchdog를 지원, 변환 종료와 상/하한 범위 이탈 인터럽트 발생
- V_{REF} : 참조 전압, STM32F103RB는 아날로그 공급 전원 V_{DDA} 와 V_{SSA} 에 연결되어 있음
- EXTRIG, JEXTRIG: ADC 변환 시작을 위한 외부 트리거, 타이머, GPIO, 인터럽트를 통해 ADC 변환 제어가 가능

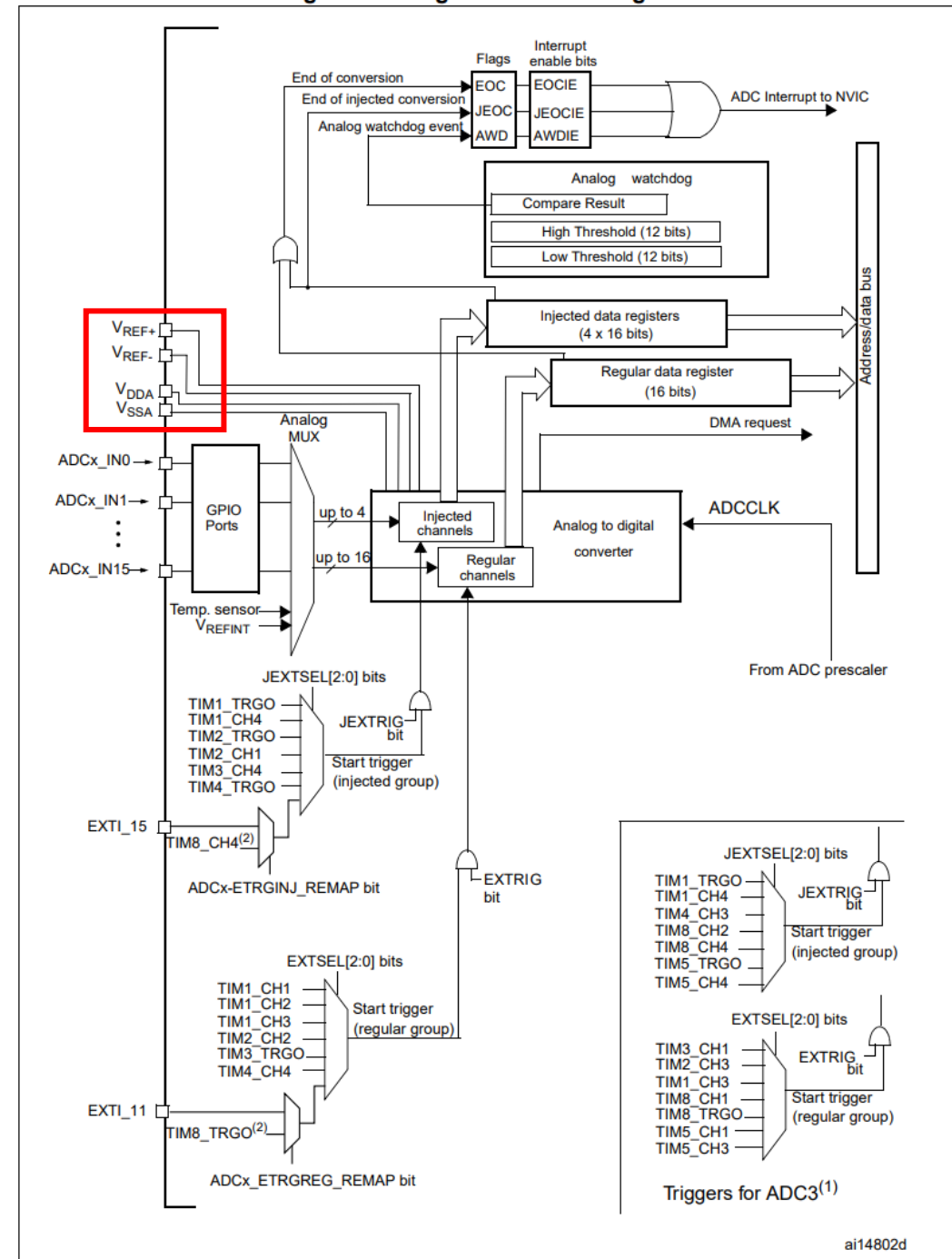
Figure 22. Single ADC block diagram



>> Analog to Digital Converter

- Data Register: 변환된 데이터 저장
- ADC Interrupt: End of conversion, End of injection conversion, Analog watchdog를 지원, 변환 종료와 상/하한 범위 이탈 인터럽트 발생
- V_{REF} : 참조 전압, STM32F103RB는 아날로그 공급 전원 V_{DDA} 와 V_{SSA} 에 연결되어 있음
- EXTRIG, JEXTRIG: ADC 변환 시작을 위한 외부 트리거, 타이머, GPIO, 인터럽트를 통해 ADC 변환 제어가 가능

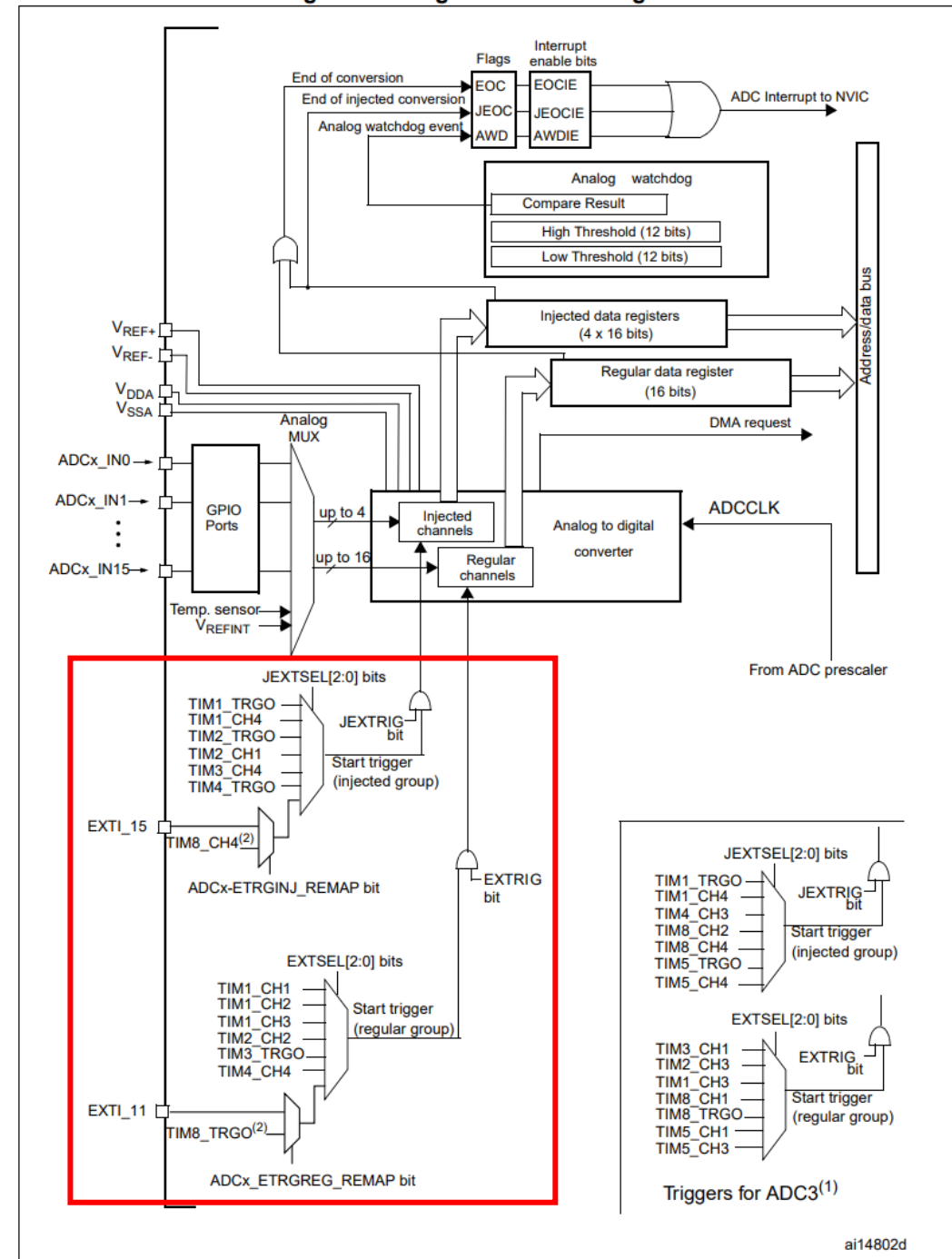
Figure 22. Single ADC block diagram



>> Analog to Digital Converter

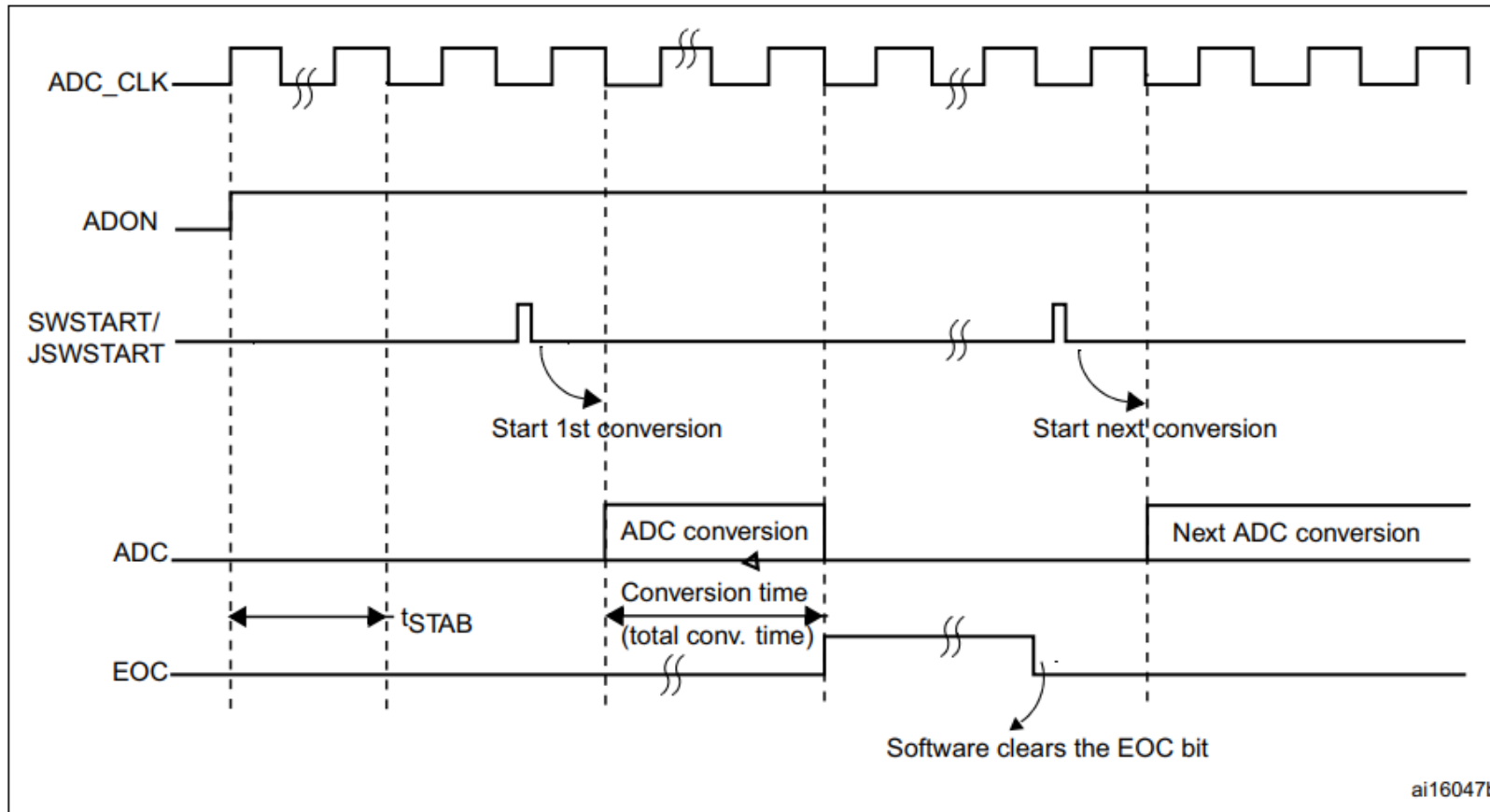
- Data Register: 변환된 데이터 저장
- ADC Interrupt: End of conversion, End of injection conversion, Analog watchdog를 지원, 변환 종료와 상/하한 범위 이탈 인터럽트 발생
- V_{REF} : 참조 전압, STM32F103RB는 아날로그 공급 전원 V_{DDA} 와 V_{SSA} 에 연결되어 있음
- EXTRIG, JEXTRIG: ADC 변환 시작을 위한 외부 트리거, 타이머, GPIO, 인터럽트를 통해 ADC 변환 제어가 가능

Figure 22. Single ADC block diagram



>> Analog to Digital Converter

Figure 23. Timing diagram



- ANON비트를 통해 ADC를 활성화, t_{STAB} 만큼의 안정화 시간을 가짐
- 변환 시작 신호를 받은 후 14클럭 주기가 지나면 EOC, 변환 완료 플래그가 set되고 16비트 ADC 데이터 레지스터에 변환된 데이터가 저장됨

>> Analog to Digital Converter

- **Single conversion mode:** 단일 변환 모드에서는 AD 변환이 한번만 발생함. ADON 비트의 활성화 혹은 외부 트리거를 통해 변환 시작. CONT 비트 0
- **Continuous conversion mode:** AD 변환이 발생한 후 다음 변환이 이어 진행됨. CONT 비트 1
 - Bit 1 **CONT:** Continuous conversion
 This bit is set and cleared by software. If set conversion takes place continuously till this bit is reset.
 0: Single conversion mode
 1: Continuous conversion mode
- **Scan mode:** 스캔 모드는 채널 그룹을 변환하기 위해 활용. SCAN 비트가 set되면 ADC_SQRx/ADC_JSQRx 레지스터를 통해 설정된 그룹 순서에 따라 변환
- **Discontinuous mode:** DISCEN 비트를 통해 설정. SQR로 선택된 채널중 $n(n \leq 8)$ 개의 연속된 채널의 변환 가능. 외부 트리거가 발생할 경우 다음 n 개의 변환을 진행하고, 변환 채널의 순서 끝에 도달하면 종료
 - $n = 3$, channels to be converted = 0, 1, 2, 3, 6, 7, 9, 10
 first trigger: sequence converted 0, 1, 2. An EOC event is generated at each conversion
 second trigger: sequence converted 3, 6, 7. An EOC event is generated at each conversion
 third trigger: sequence converted 9, 10. An EOC event is generated at each conversion
 fourth trigger: sequence converted 0, 1, 2. An EOC event is generated at each conversion

>> Analog to Digital Converter

- 채널별 샘플링 시간 설정: ADC_SMPR1,2를 통해 각 채널마다 서로 다른 샘플링 시간을 설정 가능. 긴 샘플링 시간은 보다 정확한 측정이 가능하지만 전력소비, 응답 속도에 영향

The total conversion time is calculated as follows:

$$T_{\text{conv}} = \text{Sampling time} + 12.5 \text{ cycles}$$

Example:

With an ADCCLK = 14 MHz and a sampling time of 1.5 cycles:

$$T_{\text{conv}} = 1.5 + 12.5 = 14 \text{ cycles} = 1 \mu\text{s}$$

- 외부 트리거에 의한 변환: Timer, EXTI를 트리거로 사용하여 변환 시작 제어 설정, EXTSEL/EXTJSSEL

Table 67. External trigger for regular channels for ADC1 and ADC2

Source	Type	EXTSEL[2:0]
TIM1_CC1 event	Internal signal from on-chip timers	000
TIM1_CC2 event		001
TIM1_CC3 event		010
TIM2_CC2 event		011
TIM3_TRGO event		100
TIM4_CC4 event		101
EXTI line 11 / TIM8_TRGO event ⁽¹⁾⁽²⁾	External pin / Internal signal from on-chip timers	110
SWSTART	Software control bit	111

>> Analog to Digital Converter

- ADC 예제 – 프로세서 내장 온도 아날로그 온도

- 샘플링 타임 설정

- 13.5사이클 + 12.5사이클 = 26사이클 = 3.25 μ s

The screenshot displays the STM32CubeMX configuration tool. On the left, the 'Categories' pane shows 'System Core' and 'Analog' expanded. Under 'Analog', 'ADC1' and 'ADC2' are listed. The 'Timers', 'Connectivity', 'Computing', and 'Middleware and Software Packs' sections are also visible. The main configuration area on the right shows the 'Mode' tab selected. Under 'ADCs_Common_Settings', the 'Mode' is set to 'Independent mode'. Under 'ADC_Settings', the 'Continuous Conversion Mode' is set to 'Enabled'. The 'ADC_Regular_ConversionMode' section shows 'Enable Regular Conversions' set to 'Enable', 'Number Of Conversion' set to '1', and 'External Trigger Conversion Source' set to 'Regular Conversion launched by software'. The 'Rank' is set to '1'. The 'Channel' is set to 'Channel Temperature Sensor'. The 'Sampling Time' is set to '13.5 Cycles'.



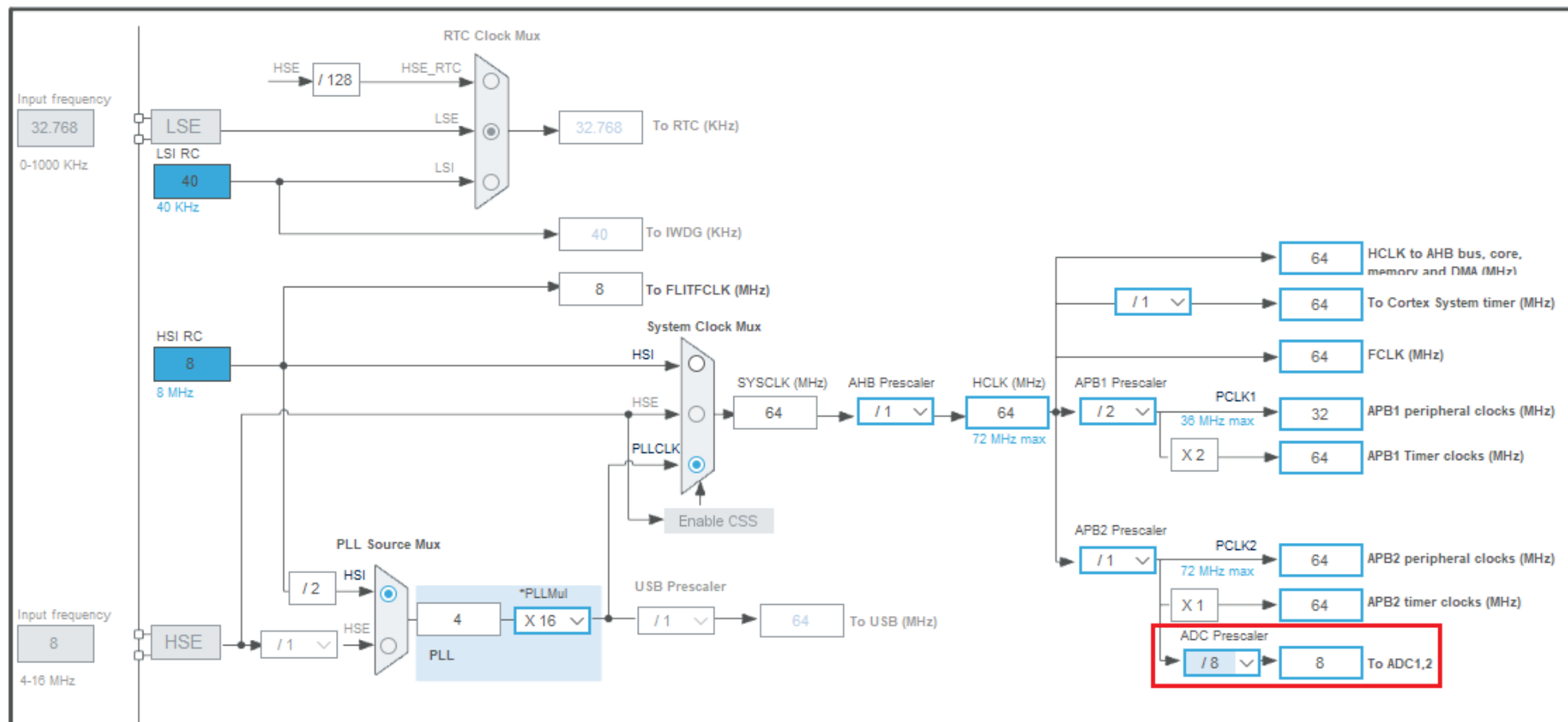
>> Analog to Digital Converter

- ADCCLK, ADC의 동작 클럭은 14MHz가 최대, 따라서 프리스케일러

ADC prescaler
/2, 4, 6, 8

ADCCLK
14 MHz max

to ADC1,2



>> Analog to Digital Converter

- UART printf 시리얼 디버깅 코드 추가
- ADC1 초기화 함수 생성

```
main.c X
51 /* Private function prototypes -----
52 void SystemClock_Config(void);
53 static void MX_GPIO_Init(void);
54 static void MX_USART2_UART_Init(void);
55 static void MX_ADC1_Init(void);
56 /* USER CODE BEGIN PFP */

161 static void MX_ADC1_Init(void)
162 {
163
164     /* USER CODE BEGIN ADC1_Init 0 */
165
166     /* USER CODE END ADC1_Init 0 */
167
168     ADC_ChannelConfTypeDef sConfig = {0};
169
170     /* USER CODE BEGIN ADC1_Init 1 */
171
172     /* USER CODE END ADC1_Init 1 */
173
174     /** Common config
175     */
176     hadcl.Instance = ADC1;
177     hadcl.Init.ScanConvMode = ADC_SCAN_DISABLE;
178     hadcl.Init.ContinuousConvMode = ENABLE;
179     hadcl.Init.DiscontinuousConvMode = DISABLE;
180     hadcl.Init.ExternalTrigConv = ADC_SOFTWARE_START;
181     hadcl.Init.DataAlign = ADC_DATAALIGN_RIGHT;
182     hadcl.Init.NbrOfConversion = 1;
183     if (HAL_ADC_Init(&hadcl) != HAL_OK)
184     {
185         Error_Handler();
186     }
187
188     /** Configure Regular Channel
189     */
190     sConfig.Channel = ADC_CHANNEL_TEMPSENSOR;
191     sConfig.Rank = ADC_REGULAR_RANK_1;
192     sConfig.SamplingTime = ADC_SAMPLETIME_13CYCLES_5;
193     if (HAL_ADC_ConfigChannel(&hadcl, &sConfig) != HAL_OK)
194     {
195         Error_Handler();
196     }
197     /* USER CODE BEGIN ADC1_Init 2 */
```

>> Analog to Digital Converter

- ADC 선언 및 초기화 이후 Calibration 진행
- 부팅 후 측정 이전에 Calibration을 통해 올바른 아날로그 입력을 읽어낼 수 있음
- Calibration을 완료한 후 HAL_ADC_START를 통해 Analog to Digital 변환 시작
- PollforConversion은 변환이 완료될때까지(혹은 Timeout 시간까지) 대기
- GetValue를 통해 Data register에서 변환한 값을 읽어옴
- 변환된 값은 12비트 해상도(0~4095)의 값이므로 16비트 unsigned int로 읽어옴

```

107  /* Infinite loop */
108  /* USER CODE BEGIN WHILE */
109
110  //ADC Calibration
111  if (HAL_ADCEx_Calibration_Start(&hadcl) != HAL_OK)
112      Error_Handler();
113
114  //ADC Start
115  if (HAL_ADC_Start(&hadcl) != HAL_OK)
116      Error_Handler();
117
118  uint16_t adcl;
119
120  while (1)
121  {
122      /* USER CODE END WHILE */
123
124      /* USER CODE BEGIN 3 */
125      HAL_ADC_PollForConversion(&hadcl, 100);
126      adcl = HAL_ADC_GetValue(&hadcl);
127      printf("ADC Read Temp: %d\n", adcl);
128
129      HAL_Delay(100);
130
131  }
132  /* USER CODE END 3 */

```

```

ADC Read Temp: 1624
ADC Read Temp: 1624
ADC Read Temp: 1622
ADC Read Temp: 1624
ADC Read Temp: 1623
ADC Read Temp: 1623
ADC Read Temp: 1622

```

>> Analog to Digital Converter

Reading the temperature

To use the sensor:

1. Select the ADCx_IN16 input channel.
2. Select a sample time of 17.1 μ s
3. Set the TSVREFE bit in the *ADC control register 2 (ADC_CR2)* to wake up the temperature sensor from power down mode.
4. Start the ADC conversion by setting the ADON bit (or by external trigger).
5. Read the resulting V_{SENSE} data in the ADC data register
6. Obtain the temperature using the following formula:

$$\text{Temperature (in } ^\circ\text{C)} = \{(V_{25} - V_{\text{SENSE}}) / \text{Avg_Slope}\} + 25.$$

Where,

V_{25} = V_{SENSE} value for 25° C and

Avg_Slope = Average Slope for curve between Temperature vs. V_{SENSE} (given in mV/° C or μ V/ °C).

Refer to the Electrical characteristics section for the actual values of V_{25} and Avg_Slope.



>> Analog to Digital Converter

- 출력된 값은 온도 센서에서 읽어낸 전압 값으로, 온도에 따라 다른 전압이 출력됨
- 레퍼런스 메뉴얼에 나타난 온도 센서의 수식과 데이터 시트에 나타난 온도 센서의 특성을 이용해 실제 온도 값을 계산할 수 있음

Temperature sensor characteristics

Table 51. TS characteristics

Symbol	Parameter	Min	Typ	Max	Unit
$T_L^{(1)}$	V_{SENSE} linearity with temperature	-	± 1	± 2	$^{\circ}\text{C}$
Avg_Slope ⁽¹⁾	Average slope	4.0	4.3	4.6	mV/ $^{\circ}\text{C}$
$V_{25}^{(1)}$	Voltage at 25 $^{\circ}\text{C}$	1.34	1.43	1.52	V
$t_{START}^{(2)}$	Startup time	4	-	10	μs
$T_{S_temp}^{(3)(2)}$	ADC sampling time when reading the temperature	-	-	17.1	

>> Analog to Digital Converter

- 수식과 값을 기반으로 코드 추가

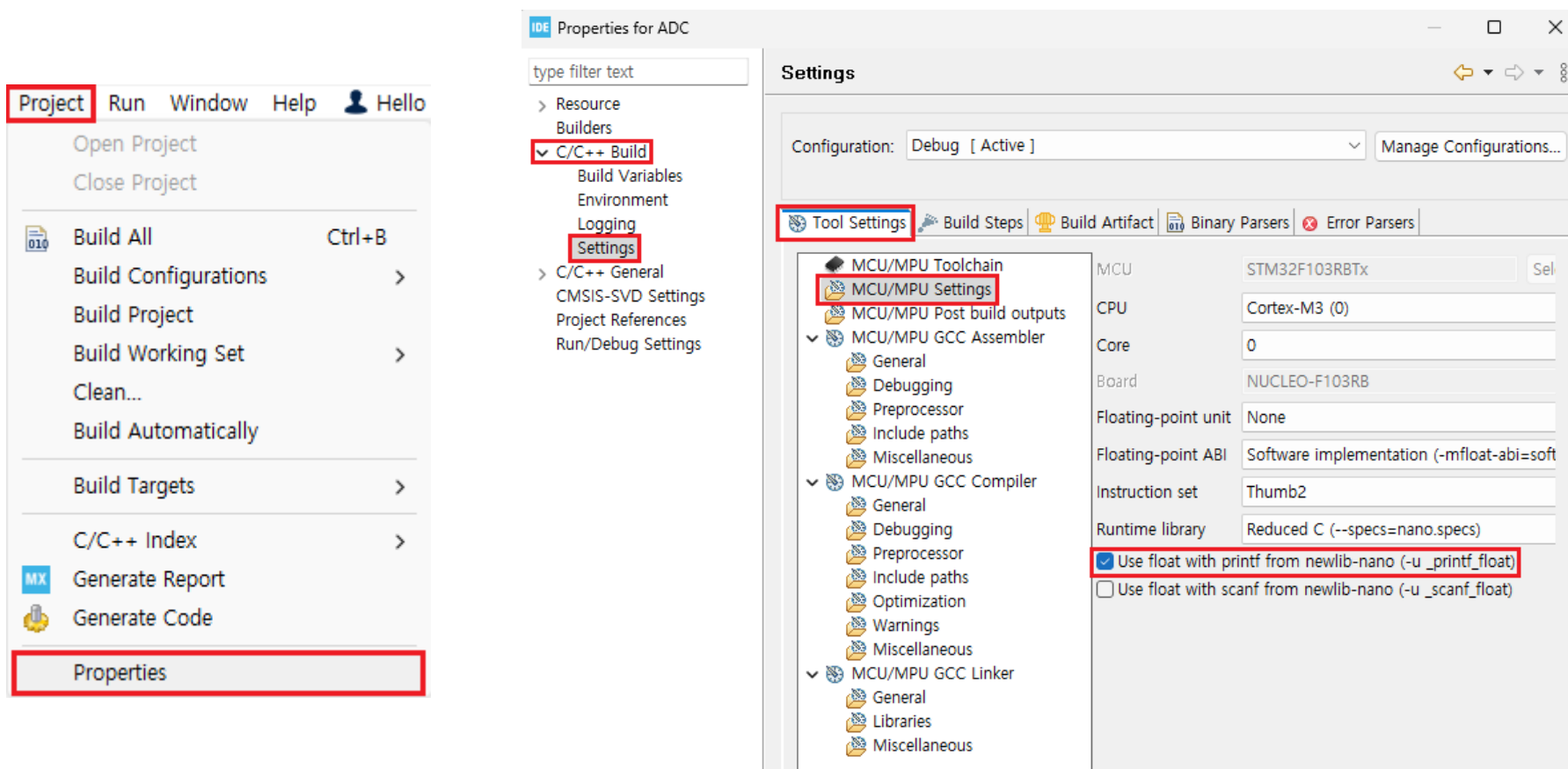
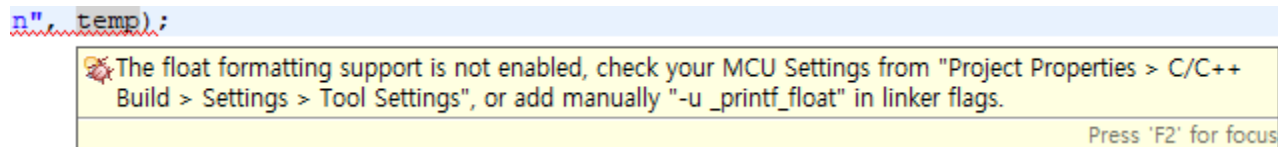
```
47 /* USER CODE BEGIN PV */
48 const float V25      = 1.43;
49 const float AvgSlope = 4.3E-3;
50 const float ADC_V     = 3.3;
51 /* USER CODE END PV */

126 uint16_t adcl;
127
128 float Vsense;
129 float temp;
130
131 while (1)
132 {
133     /* USER CODE END WHILE */
134
135     /* USER CODE BEGIN 3 */
136     HAL_ADC_PollForConversion(&hadcl, 100);
137     adcl = HAL_ADC_GetValue(&hadcl);
138     // printf("ADC Read Temp: %d\n", adcl);
139
140     Vsense = adcl * ADC_V / 4096;
141     temp = (V25 - Vsense) / AvgSlope + 25;
142     printf("Temp: %f\n", temp);
143
144     HAL_Delay(100);
145
146 }
147 /* USER CODE END 3 */
```



>> Analog to Digital Converter

- printf에서 float 자료형 출력으로 인한 경고 해결



>> Analog to Digital Converter

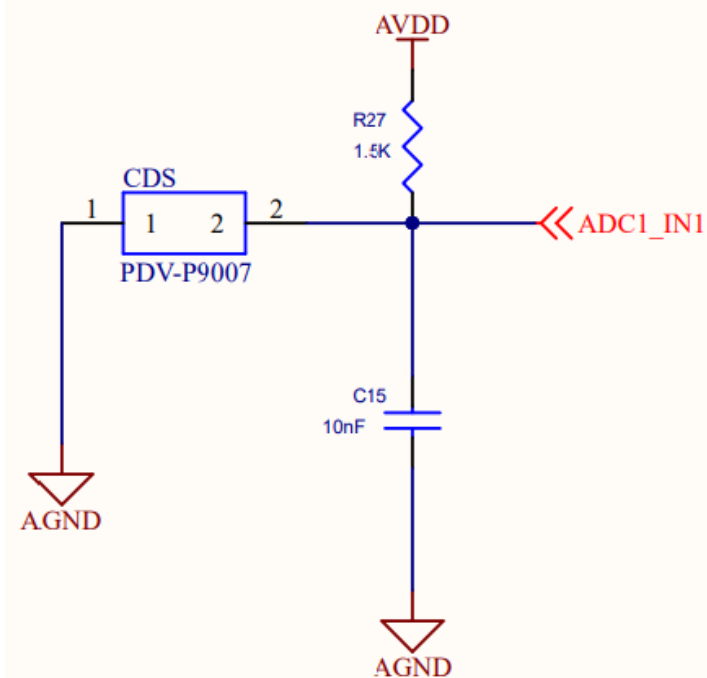
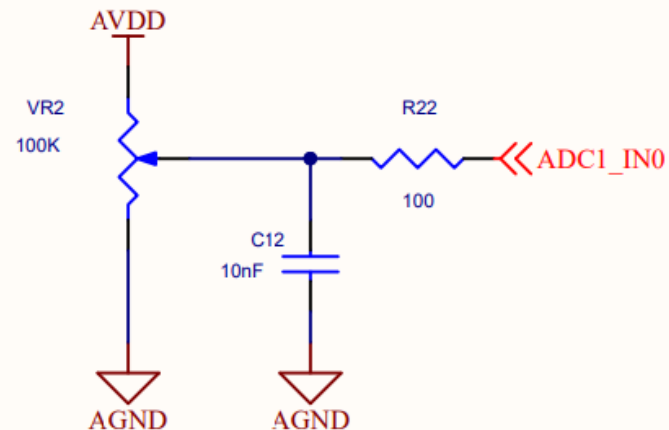
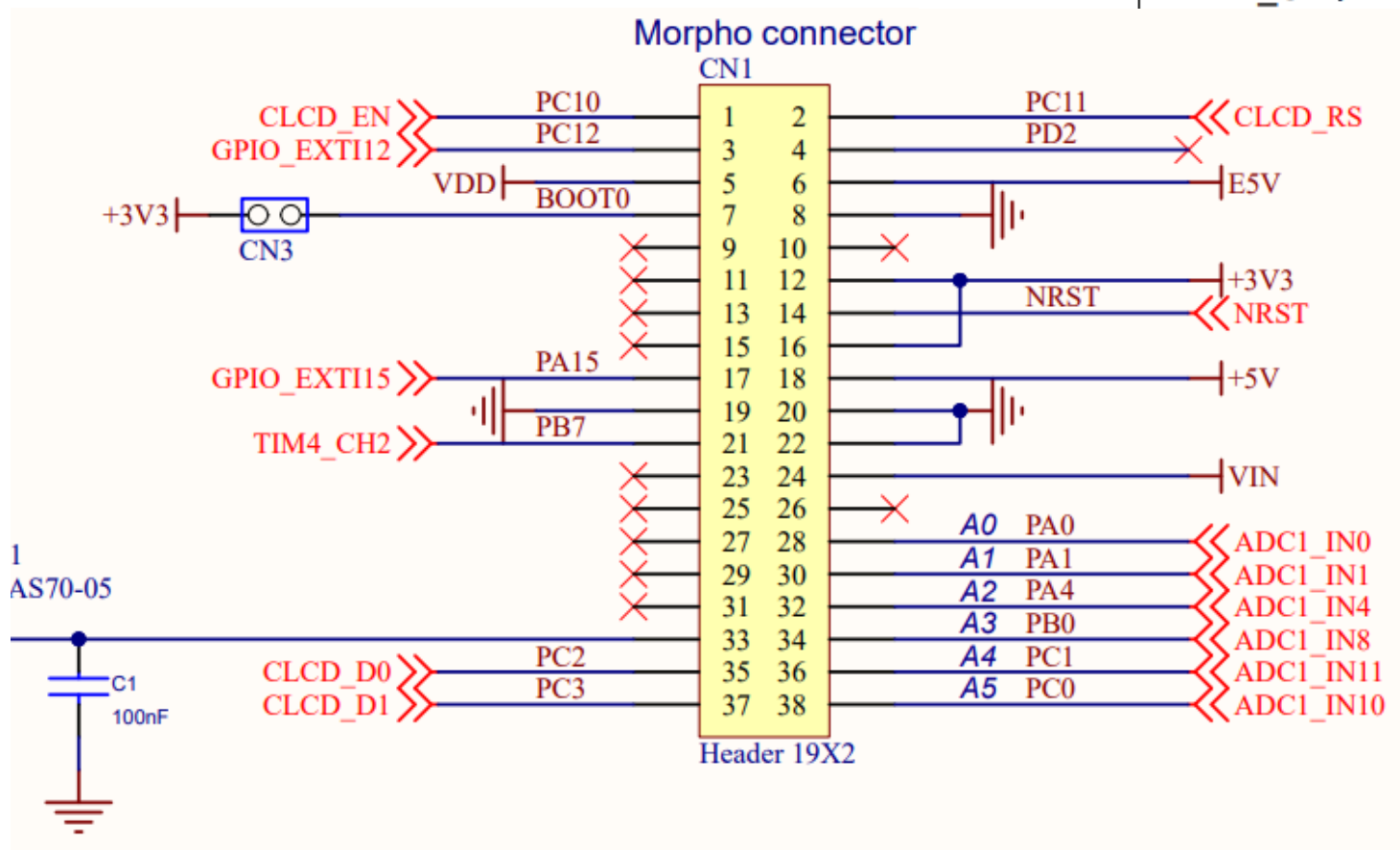
- printf 설정 적용후 동작 확인
- ADC_CLK: 8MHz, 13.5Cycles -> 1.68 μ s
- 따라서 레퍼런스 메뉴얼에 적합한 ADC_CLK과 Sampling time(Cycles)로 코드 수정해보기

```
COM6 - Tera Term VT
메뉴(F) 수정(E) 설정(S) 제어(O) 창(W) 도움말(H)
Temp: 49.906868
Temp: 49.719513
Temp: 49.719513
Temp: 49.906868
Temp: 49.906868
Temp: 49.906868
Temp: 49.906868
Temp: 49.719513
Temp: 50.094246
Temp: 50.094246
Temp: 49.719513
Temp: 49.906868
Temp: 49.906868
Temp: 49.906868
Temp: 50.094246
Temp: 49.906868
Temp: 49.906868
Temp: 49.906868
Temp: 50.094246
Temp: 50.281601
Temp: 49.906868
Temp: 50.281601
Temp: 50.094246
Temp: 50.094246
Temp: 50.094246
Temp: 50.094246
```

>> Analog to Digital Converter

- 확장 보드 실습 - 가변저항/조도센서

PA0	WKUP/ USART2_CTS ⁽⁹⁾ / ADC12_IN0/ TIM2_CH1_ ETR ⁽⁹⁾
PA1	USART2_RTS ⁽⁹⁾ / ADC12_IN1/ TIM2_CH2 ⁽⁹⁾



>> Analog to Digital Converter

ADC1 Mode and Configuration

Mode

☒ IN0☒ IN1☐ IN2☐ IN3☐ IN4☐ IN5

Configuration

Reset Configuration

☒ NVIC Settings☒ DMA Settings☒ GPIO Settings☒ Parameter Settings☒ User Constants

Configure the below parameters :

Search (Ctrl+F)

ADCs_Common_Settings

Mode

Independent mode

ADC_Settings

Data Alignment

Right alignment

Scan Conversion Mode Enabled

Continuous Conversion Mode Enabled

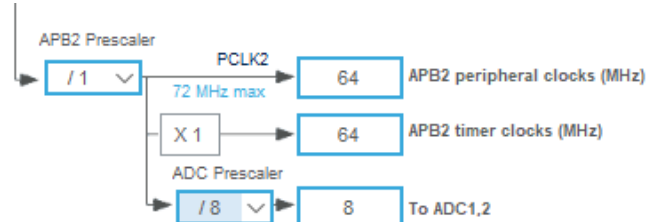
Discontinuous Conversion Mode Disabled

ADC_Regular_ConversionMode

Enable Regular Conversions Enable

Number Of Conversion 2

External Trigger Conversion So... Regular Conversion launched by software



ADC_Regular_ConversionMode

Enable Regular Conversions Enable

Number Of Conversion 2

External Trigger Conversion Source Regular Conversion launched by software

Rank 1

Channel Channel 0

Sampling Time 13.5 Cycles

Rank 2

Channel Channel 1

Sampling Time 13.5 Cycles

Configuration

Group By Peripherals

☒ ADC☒ SYS☒ USART☒ NVIC☒ GPIO☒ Single Mapped Signals

Search Signals

Search (Ctrl+F)

☐ Show only Modified Pins

Pin N...	Signal o...	GPI...	GPIO m...	GPI...	Maxi...	User Label	Modified
PA0-WK...	ADC1_IN0	n/a	Analog ...	n/a	n/a	ADC1_IN0_VAR	☒
PA1	ADC1_IN1	n/a	Analog ...	n/a	n/a	ADC1_IN1_CDS	☒



>> Analog to Digital Converter

Scan mode

This mode is used to scan a group of analog channels.

Scan mode can be selected by setting the SCAN bit in the ADC_CR1 register. Once this bit is set, ADC scans all the channels selected in the ADC_SQRx registers (for regular channels) or in the ADC_JSQR (for injected channels). A single conversion is performed for each channel of the group. After each end of conversion the next channel of the group is converted automatically. If the CONT bit is set, conversion does not stop at the last selected group channel but continues again from the first selected group channel.

When using scan mode, DMA bit must be set and the direct memory access controller is used to transfer the converted data of regular group channels to SRAM after each update of the ADC_DR register.

The injected channel converted data is always stored in the ADC_JDRx registers.

DMA1

MemToMem

DMA Request	Channel	Direction	Priority
ADC1	ADC1	Peripheral To Memory	Low

Add

Delete

DMA Request Settings

Mode

Circular

Increment Address

☐

Data Width

Half Word

Peripheral

Memory

☐

☒

Half Word

Half Word

>> Analog to Digital Converter

- ADC Poll for Conversion를 활용해 ADC 값을 읽어올 수 있으나, 읽어오는 타이밍이 너무 빠를 경우 값이 덮어 씌워지는 문제가 발생할 수 있음
- Scan 모드로 여러 채널의 ADC 값을 읽을 때는 레퍼런스 매뉴얼의 권장 방법인 Direct Memory Access를 통해 읽는 것이 좋음
- 폴링의 경우 EOC 플래그를 지속적으로 확인해야 하므로 프로세서 부하가 존재, DMA의 경우 프로세서를 거치지 않으므로 프로세서 부하가 최소화
- DMA는 주기적으로 데이터 메모리로 바로 전송하여, 인터럽트 등으로 인한 변환 사이클이나 지연 시간의 변화에 안정적임

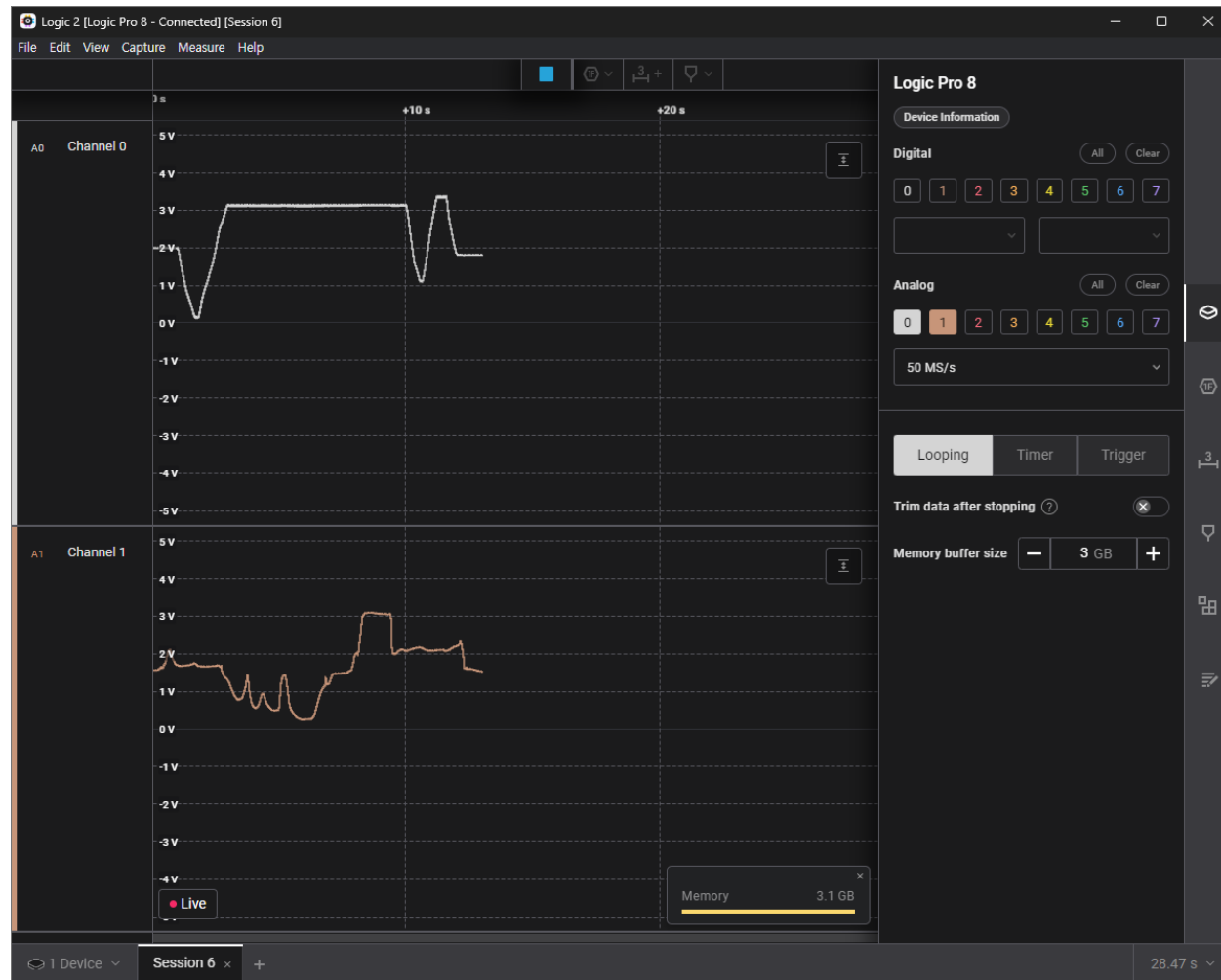
```

119  uint16_t ADC_Val[2];
120
121  if (HAL_ADCEx_Calibration_Start(&hadcl) != HAL_OK)
122      Error_Handler();
123
124  if (HAL_ADC_Start_DMA(&hadcl, (uint32_t*)ADC_Val, 2) != HAL_OK)
125      Error_Handler();
126
127  while (1)
128  {
129      /* USER CODE END WHILE */
130
131      /* USER CODE BEGIN 3 */
132
133      printf("VAR: %4d | CDS: %4d\n", ADC_Val[0], ADC_Val[1]);
134      HAL_Delay(100);
135
136  }
137  /* USER CODE END 3 */

```



>> Analog to Digital Converter



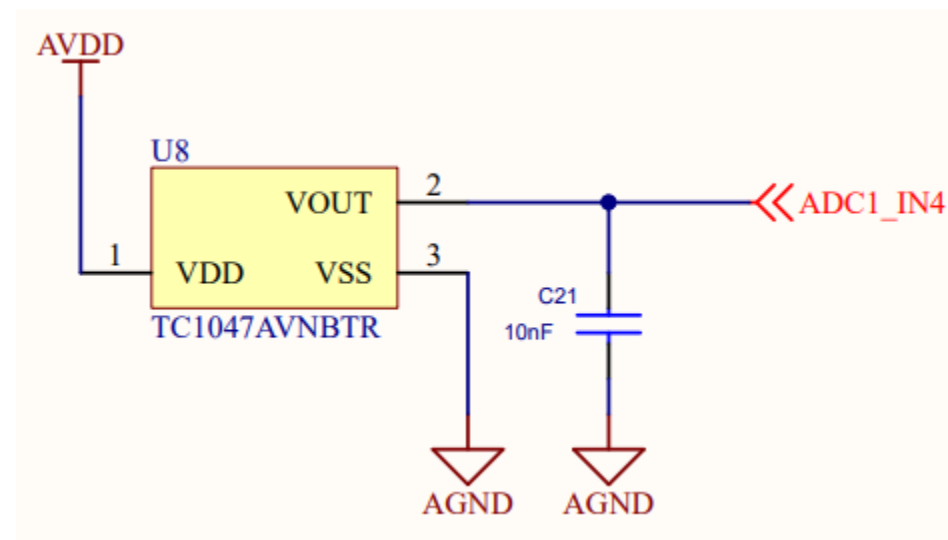
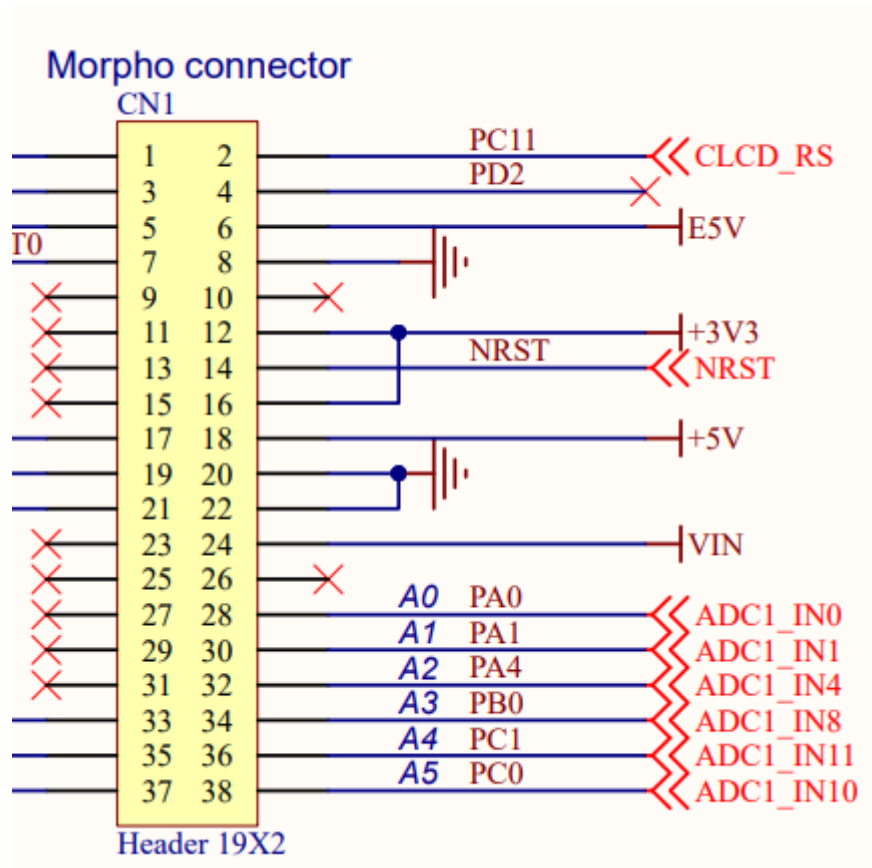
COM6 - Tera Term VT

메뉴(F) 수정(E) 설정(S) 제어(O) 창(W) 도움말(H)

VAR: 644	CDS: 8192
VAR: 2410	CDS: 1955
VAR: 2408	CDS: 1879
VAR: 2409	CDS: 1983
VAR: 2408	CDS: 2515
VAR: 2405	CDS: 2072
VAR: 1414	CDS: 2030
VAR: 509	CDS: 2069
VAR: 158	CDS: 2016
VAR: 1185	CDS: 1996
VAR: 2058	CDS: 2011
VAR: 3218	CDS: 2024
VAR: 3817	CDS: 1486
VAR: 3816	CDS: 958
VAR: 3816	CDS: 1163
VAR: 3815	CDS: 781
VAR: 3813	CDS: 818
VAR: 3813	CDS: 777
VAR: 3811	CDS: 556
VAR: 3817	CDS: 1635
VAR: 3812	CDS: 651
VAR: 3811	CDS: 305
VAR: 3810	CDS: 252
VAR: 3811	CDS: 320
VAR: 3818	CDS: 1153
VAR: 3817	CDS: 1441
VAR: 3820	CDS: 1769
VAR: 3822	CDS: 1787
VAR: 3819	CDS: 1881
VAR: 3819	CDS: 2559
VAR: 3824	CDS: 3765
VAR: 3824	CDS: 3762
VAR: 3824	CDS: 3737
VAR: 3823	CDS: 3730
VAR: 3820	CDS: 2515
VAR: 3804	CDS: 2508
VAR: 2007	CDS: 2604
VAR: 1303	CDS: 2616
VAR: 2689	CDS: 2521
VAR: 4093	CDS: 2537
VAR: 4093	CDS: 2519
VAR: 2788	CDS: 2624
VAR: 2185	CDS: 2835
VAR: 2181	CDS: 1916
VAR: 2177	CDS: 1879

>> Analog to Digital Converter

- 확장 보드 실습 - 온도 센서 TC1047



>> Analog to Digital Converter

Parameter Settings

User Constants

NVIC Settings

DMA Settings

GPIO Settings

Configure the below parameters :

Search (Ctrl+F)

◀▶

i

ADCs_Common_Settings

ModeIndependent mode

ADC_Settings

Data AlignmentRight alignment

Scan Conversion ModeDisabled

Continuous Conversion ModeEnabled

Discontinuous Conversion ModeDisabled

ADC_Regular_ConversionMode

Enable Regular ConversionsEnable

Number Of Conversion1

External Trigger Conversion SourceRegular Conversion launched by software

Rank1

ChannelChannel 4

Sampling Time239.5 Cycles

ADC_Injected_ConversionMode

Enable Injected ConversionsDisable

WatchDog

Enable Analog WatchDog Mode☐

Parameter Settings

User Constants

NVIC Settings

DMA Settings

GPIO Settings

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
ADC1 and ADC2 global interrupts	☒	0	0



>> Analog to Digital Converter

- AD 변환이 완료되면 ADC1_2 인터럽트가 호출되어 ConvCpltCallback 함수가 호출

```
stm32f1xx_it.c X
195 /* STM32F1xx Peripheral Interrupt Handlers
196 /* Add here the Interrupt Handlers for the used peripherals.
197 /* For the available peripheral interrupt handler names,
198 /* please refer to the startup file (startup_stm32f1xx.s).
199 /******
200
201 /**
202  * @brief This function handles ADC1 and ADC2 global interrupts.
203  */
204 void ADC1_2_IRQHandler(void)
205 {
206     /* USER CODE BEGIN ADC1_2_IRQn 0 */
207
208     /* USER CODE END ADC1_2_IRQn 0 */
209     HAL_ADC_IRQHandler(&hadc1);
210     /* USER CODE BEGIN ADC1_2_IRQn 1 */
211
212     /* USER CODE END ADC1_2_IRQn 1 */
213 }
214

stm32f1xx_hal_adc.c X
1908 /*
1909 __weak void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
1910 {
1911     /* Prevent unused argument(s) compilation warning */
1912     UNUSED(hadc);
1913     /* NOTE : This function should not be modified. When the callback is needed,
1914             function HAL_ADC_ConvCpltCallback must be implemented in the user file.
1915     */
1916 }
```



>> Analog to Digital Converter

```

47 /* USER CODE BEGIN PV */
48 uint16_t ADC_Temp_V;
49
50 /* USER CODE END PV */
51
52 /* Private function prototypes -----
53 void SystemClock_Config(void);
54 static void MX_GPIO_Init(void);
55 static void MX_USART2_UART_Init(void);
56 static void MX_ADC1_Init(void);
57 /* USER CODE BEGIN PFP */
58 #ifdef __GNUC__
59 #define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
60 #else
61 #define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
62 #endif
63
64 PUTCHAR_PROTOTYPE
65 {
66     if (ch == '\n')
67         HAL_UART_Transmit(&huart2, (uint8_t*) "\r", 1, 0xFFFF);
68     HAL_UART_Transmit(&huart2, (uint8_t*) &ch, 1, 0xFFFF);
69
70     return ch;
71 }
72 /* USER CODE END PFP */

```

```

116
117 if (HAL_ADCEx_Calibration_Start(&hadc1) != HAL_OK)
118     Error_Handler();
119
120 HAL_ADC_Start_IT(&hadc1);
121
122 while (1)
123 {
124     /* USER CODE END WHILE */
125     printf("Temp: %d\n", ADC_Temp_V);
126     HAL_Delay(100);
127     /* USER CODE BEGIN 3 */
128 }
129 /* USER CODE END 3 */
130 }

```

```

298 /* USER CODE BEGIN 4 */
299 void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc) {
300     ADC_Temp_V = HAL_ADC_GetValue(hadc);
301 }
302 /* USER CODE END 4 */

```

```

Temp: 948
Temp: 949
Temp: 946
Temp: 948
Temp: 948
Temp: 951
Temp: 945

```



>> Analog to Digital Converter

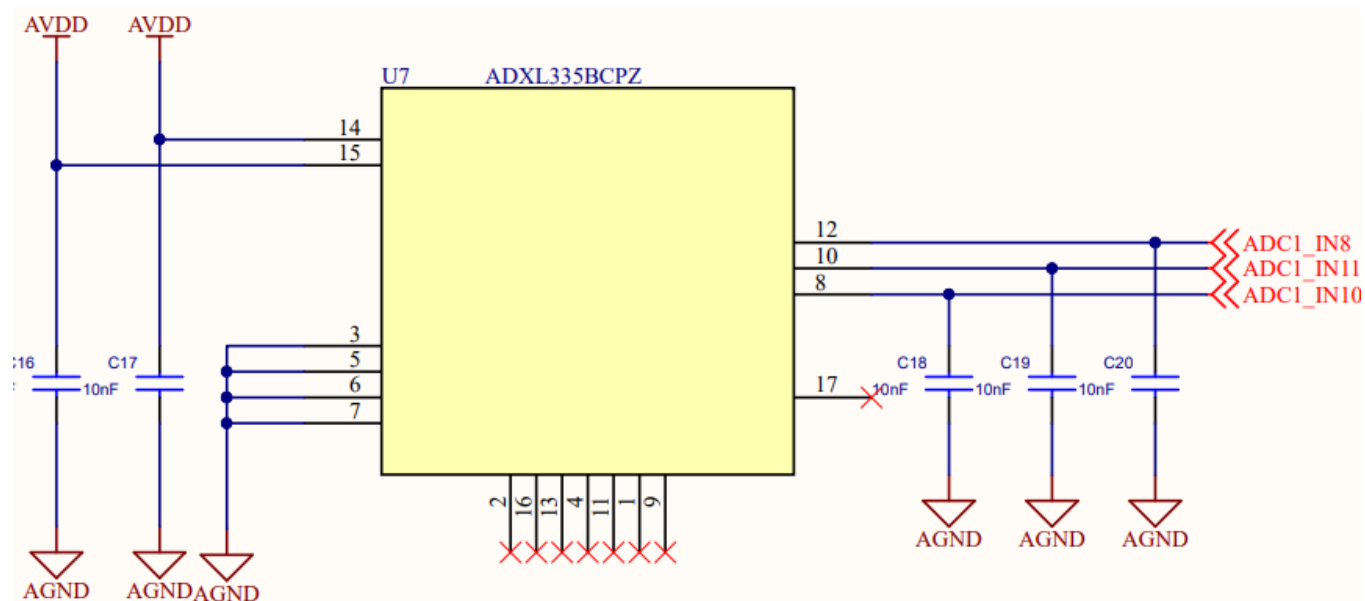
- ADC 실습 과제 1

- TC1047 온도 센서의 데이터시트를 검색 및 참고하여 현재 온도를 UART printf로 출력
- 동시에 가변저항의 AD값을 이용하여 LED 제어
- 가변저항의 AD 값을 구간으로 나누어 낮은 값부터 높은 값까지 변함에 따라 R -> G -> B -> Y LED를 순차적으로 켜기

>> Analog to Digital Converter

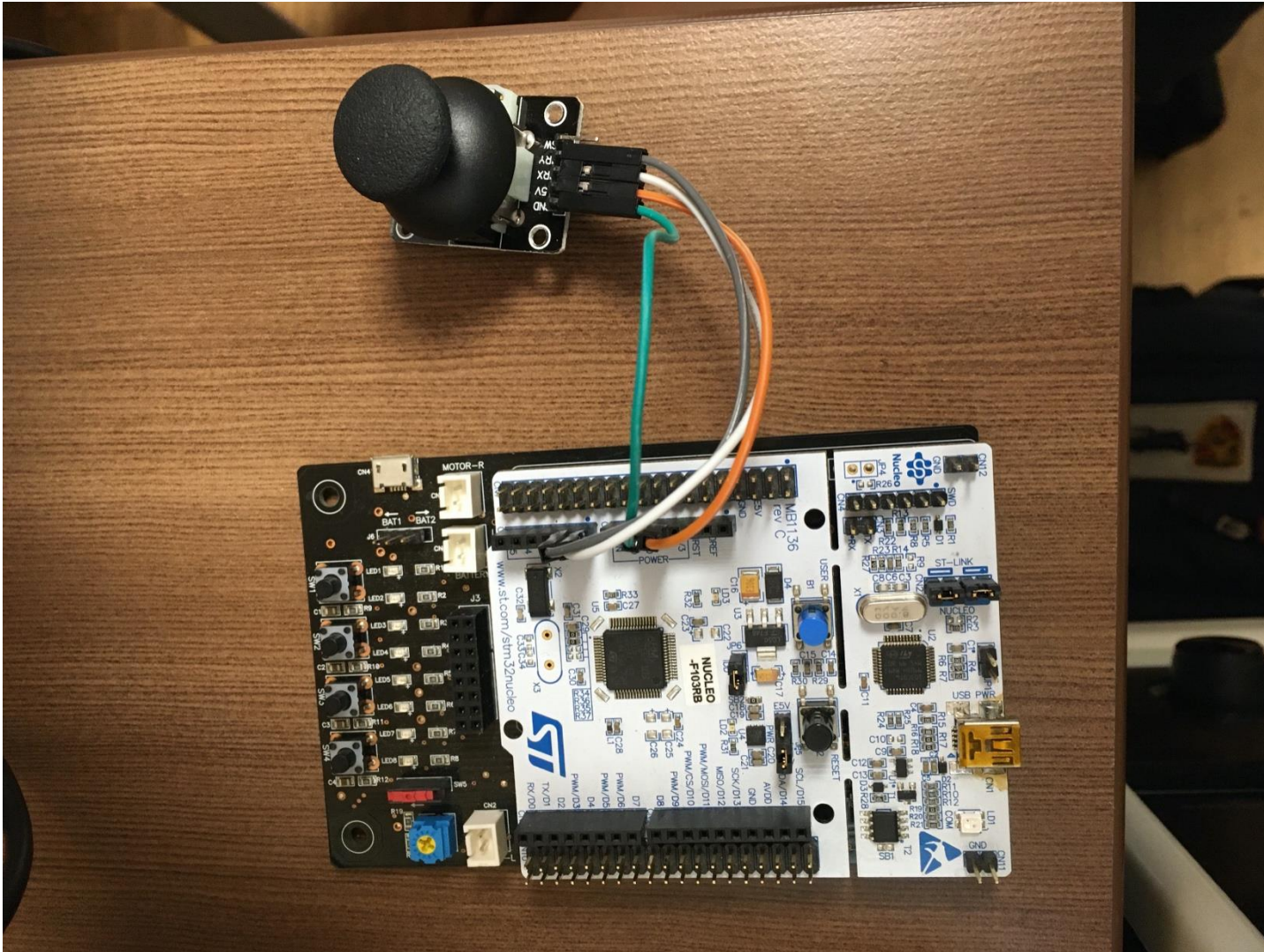
• ADC 실습 과제 2

- 3축 가속도 센서의 x, y, z 축의 가속도 측정하기
- 0g일때 1.65V가 측정되며, 센서의 감도는 300mV/g
- 측정된 가속도를 UART printf로 출력



추가. ADC 예제

Multi-channel ADC-DMA 예제



- XBOX나 PS에서 사용하는 아날로그 조이스틱임



Yikeshu Store

99.1% 긍정적인 평가

+ Follow
1473 팔로워

...을 사고 싶습니다.

Yikeshu Store

매장 홈 제품 ▾ 새일 아이템 최고 매출 신상품 피드백

고품질 이중 축 xy 조이스틱 모듈 ps2 조이스틱 제어 센서 arduino KY-02

★★★★★ 5.0~ 22 리뷰 69 주문

US \$0.27

US\$0.34 -20%

수량: 1

추가 1% 할인(10 개 이상)
9641 개 사용 가능

배송: US \$0.82
Cainiao Super Economy for Special Goods(특) 통해 Korea(으)로
예상 배송 날짜: 30-50일

즉시 구매

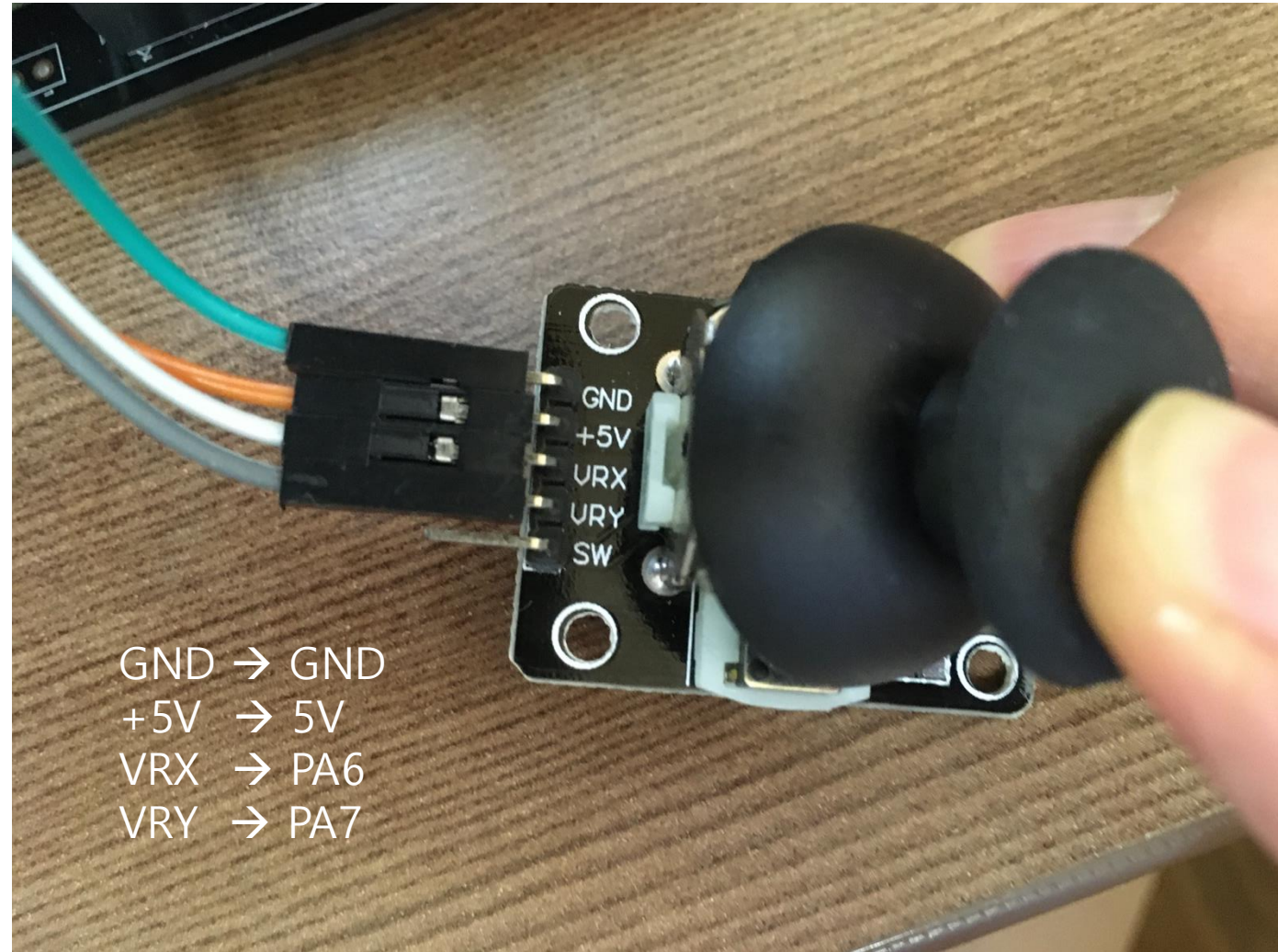
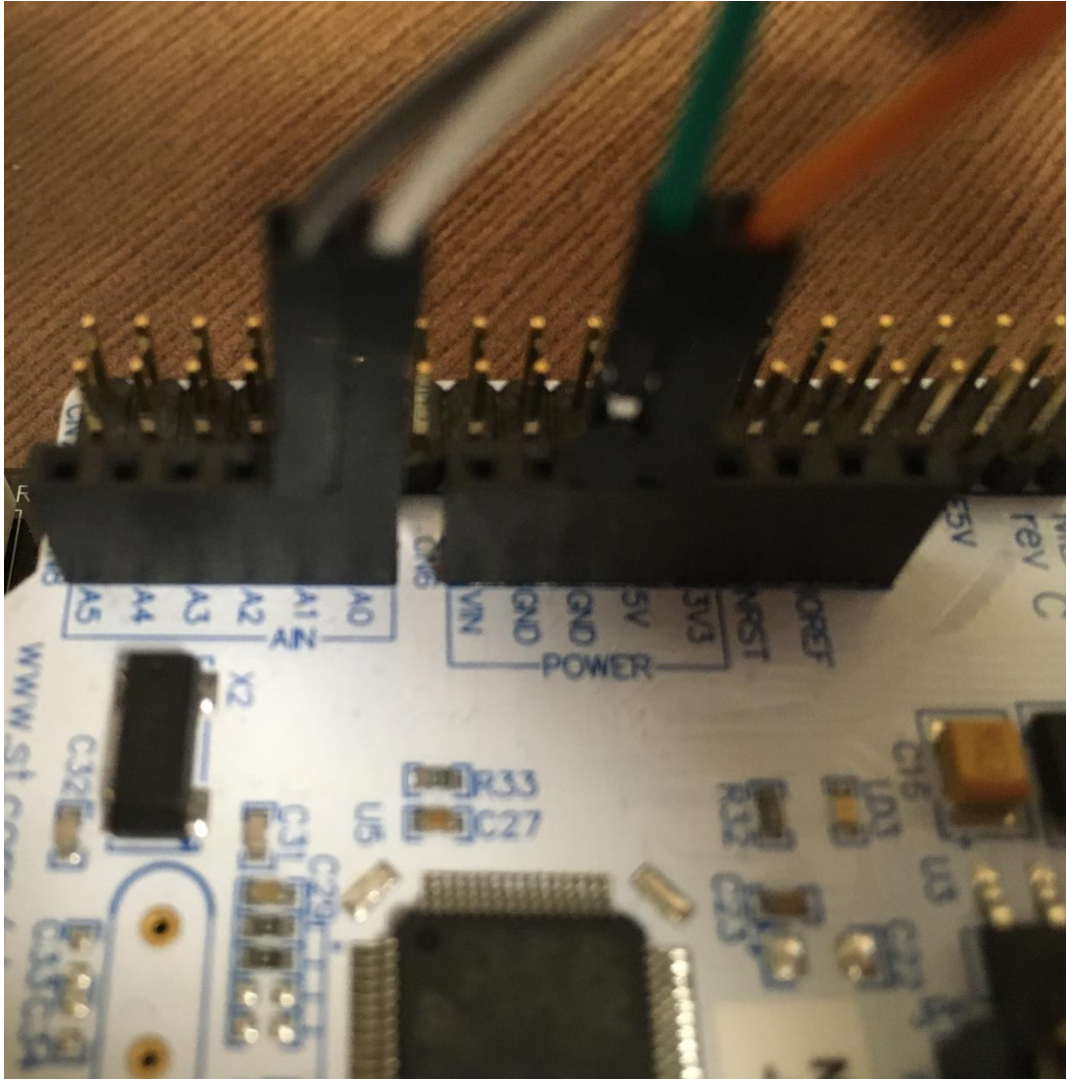
카트에 넣기

51

90일 구매자 보호
결제 금액 환불 보증

추가. ADC 예제

Multi-channel ADC-DMA 예제

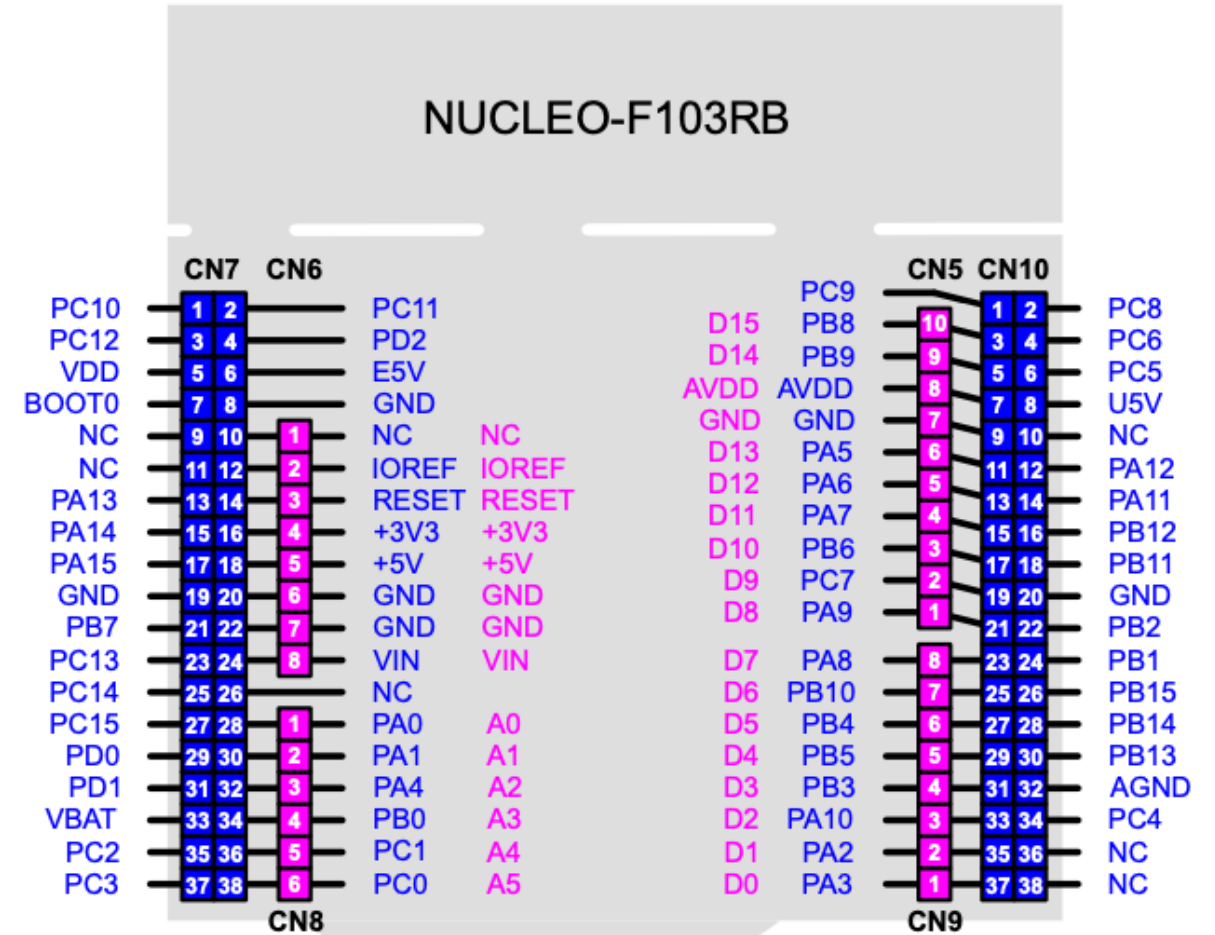
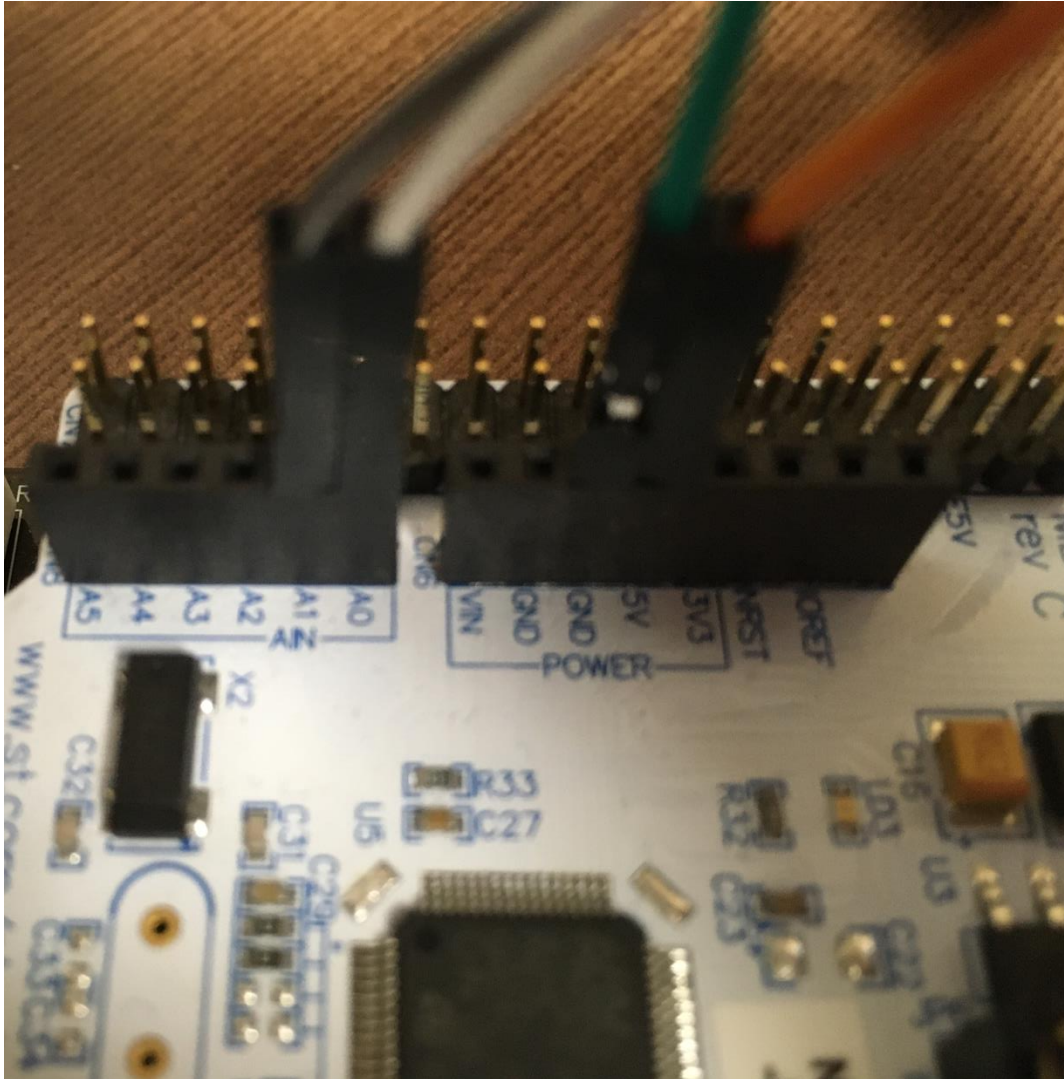


GND → GND
+5V → 5V
VRX → PA6
VRY → PA7

나눠준 조이스틱 마다 핀의 순서는 다를 수 있음

추가. ADC 예제

Multi-channel ADC-DMA 예제



조이스틱 → 커넥터 → ARM 핀
 GND → GND → GND
 +5V → +5V → +5V
 VRX → ADC1_IN6 → PA6
 VRY → ADC1_IN7 → PA7

>> Analog to Digital Converter

Mode

IN5

IN6

IN7

IN8

IN9

Configuration

Reset Configuration

NVIC Settings

DMA Settings

GPIO Settings

Parameter Settings

User Constants

Configure the below parameters :

Search (Ctrl+F)

ADC_Settings

Data Alignment

Right alignment

Scan Conversion Mo...

Enabled

Continuous Conver...

Enabled

Discontinuous Con...

Disabled

ADC_Regular_Conversion...

Enable Regular Con...

Enable

Number Of Convers...

2

External Trigger Co...

Regular Conversion launched ...

ADC_Regular_Conversion...

Enable Regular Con...

Enable

Number Of Convers...

2

External Trigger Co...

Regular Conversion launched ...

Rank

1

Channel

Channel 6

Sampling Time

13.5 Cycles

Rank

2

Channel

Channel 7

Sampling Time

13.5 Cycles

DMA Request	Channel	Direction	Priority
ADC1	ADC1	Peripheral To Memory	Low

AddDelete

DMA Request Settings

Mode

Circular

Increment Address

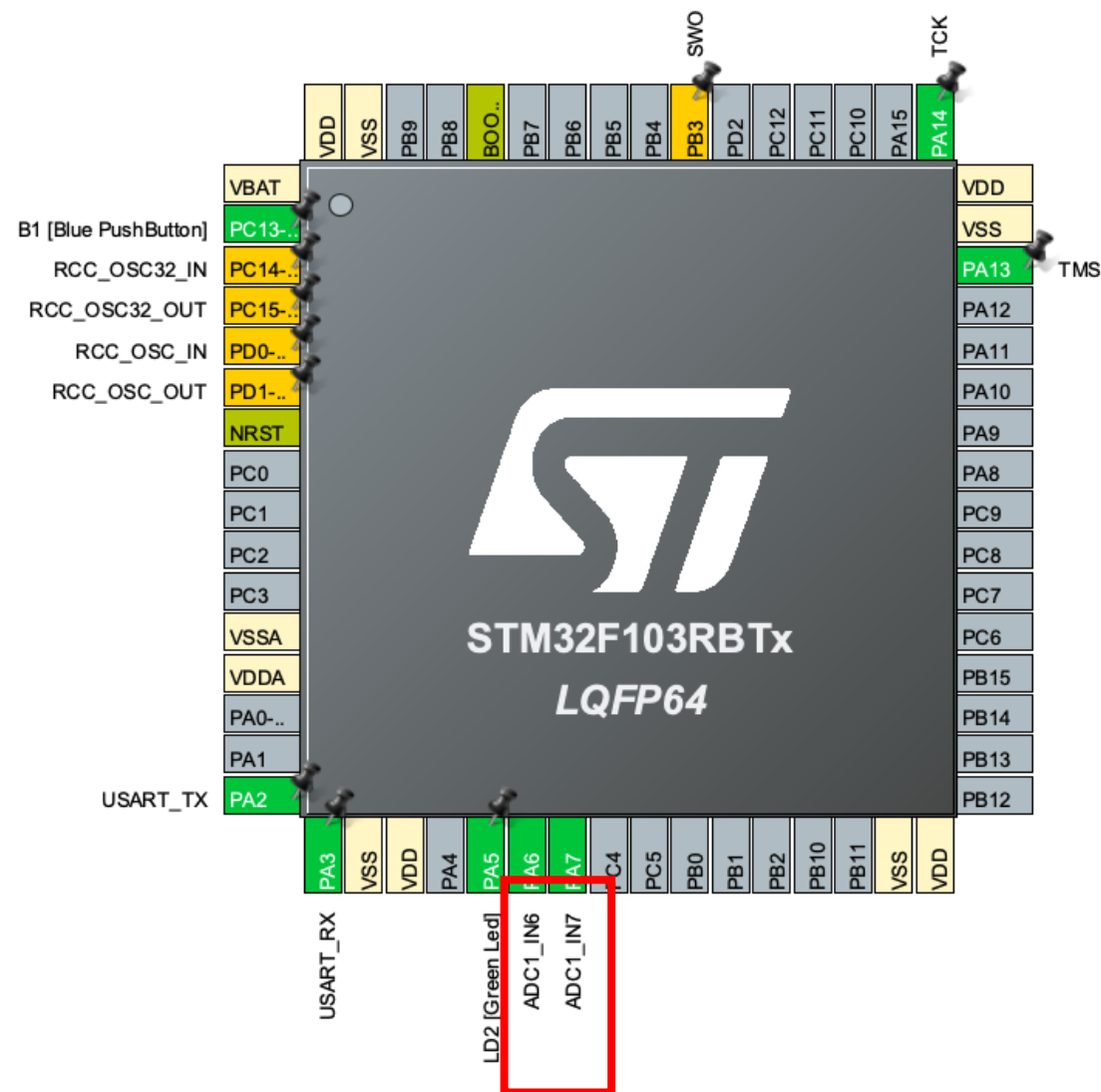
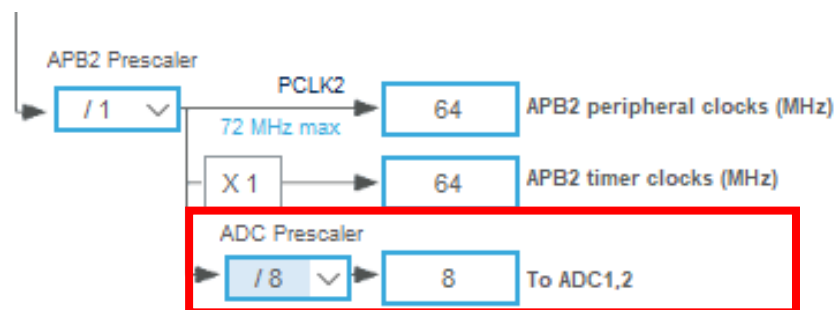
Data Width

Half Word

Peripheral

Memory

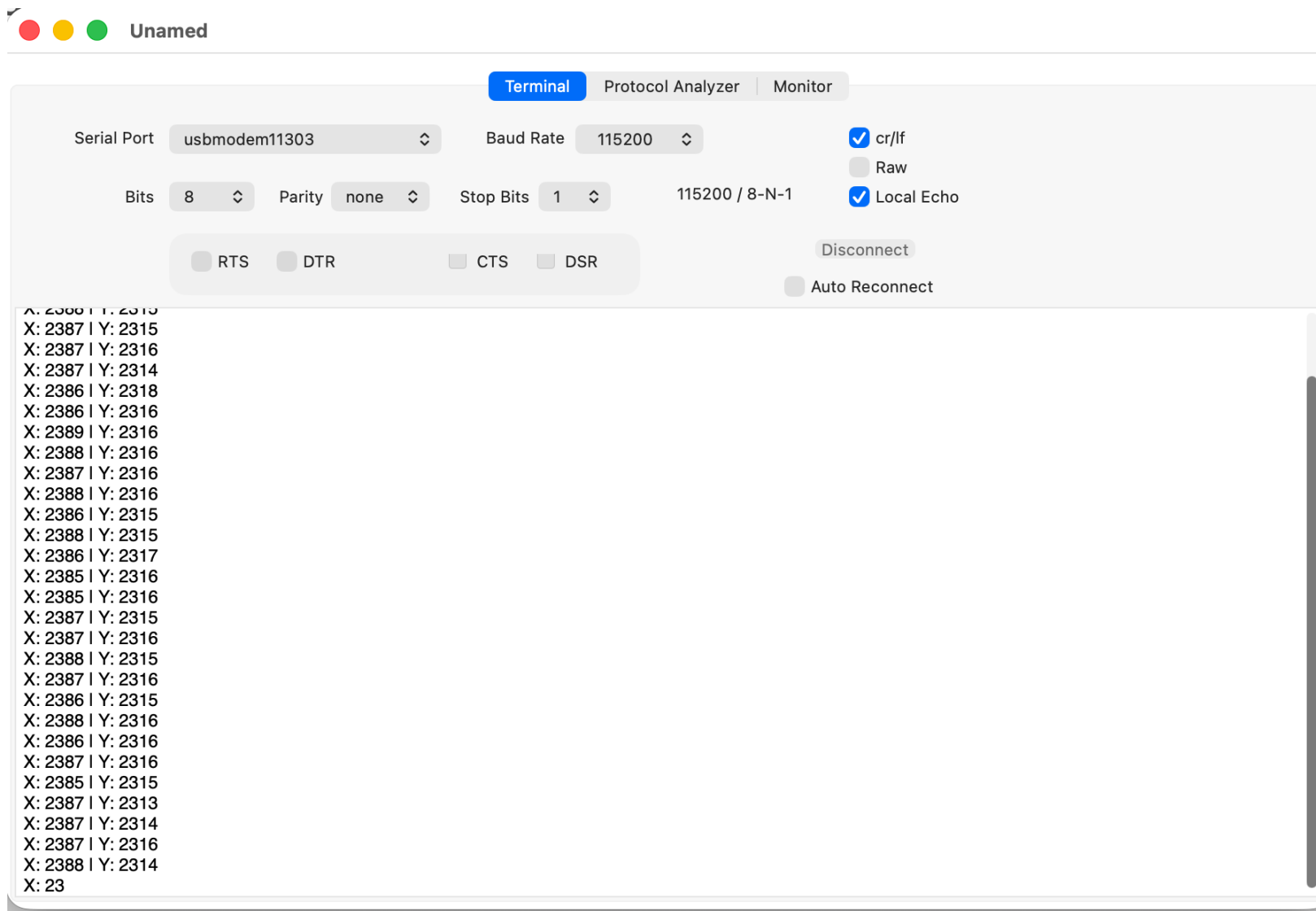
>> Analog to Digital Converter



>> Analog to Digital Converter

- ADC Poll for Conversion를 활용해 ADC 값을 읽어올 수 있으나, 읽어오는 타이밍이 너무 빠를 경우 값이 덮어 씌워지는 문제가 발생할 수 있음
- Scan 모드로 여러 채널의 ADC 값을 읽을 때는 레퍼런스 매뉴얼의 권장 방법인 Direct Memory Access를 통해 읽는 것이 좋음
- 폴링의 경우 EOC 플래그를 지속적으로 확인해야 하므로 프로세서 부하가 존재, DMA의 경우 프로세서를 거치지 않으므로 프로세서 부하가 최소화
- DMA는 주기적으로 데이터 메모리로 바로 전송하여, 인터럽트 등으로 인한 변환 사이클이나 지연 시간의 변화에 안정적임

```
117  /* USER CODE BEGIN WHILE */
118  uint16_t ADC_Val[2];
119
120  if (HAL_ADCEx_Calibration_Start(&hadc1) != HAL_OK)
121      Error_Handler();
122
123  if (HAL_ADC_Start_DMA(&hadc1, (uint32_t*)ADC_Val, 2) != HAL_OK)
124      Error_Handler();
125
126  while (1)
127  {
128      /* USER CODE END WHILE */
129
130      /* USER CODE BEGIN 3 */
131      printf("X: %4d | Y: %4d\n", ADC_Val[0], ADC_Val[1]);
132      HAL_Delay(100);
133  }
134  /* USER CODE END 3 */
135  }
136
```

Thank you

Department of Electronic Engineering,
Kyung Hee University, 1732, Deogyong-Daero,
Giheung-gu, Yongin-si, Gyeonggi-Do, 17104,
Republic of Korea

Web: <https://khu-hri.weebly.com/>



Human-Robot Interaction
Laboratory



KYUNG HEE
UNIVERSITY